FIDELITYLOOP: SUPPLEMENTARY EVIDENCE AND REPRODUCTION

A REPRODUCTION AND CLAIM–EVIDENCE MAP

The source release includes synthetic interface checks, historical facade compatibility checks, and a standalone same-runtime accounting verifier. `REPRODUCING.md` gives the commands and expected outputs. In the same-runtime comparison package, `python -B verify.py` checks the manifest and recomputes 169 resource/cost values from reduced records. Its ten tests run with `python -B -m unittest`; physical-run acceptance is inherited.

The paired trace corpus[1] covers the held-out, mixed-policy, deployment, screening, engine, arrival, and 14B studies: 197 technically accepted physical runs (including 41 strict P1+ failures) and 239 unique frozen predictions form 555 model/run pairs. Predictions are reused across physical repeats. The index separately lists eight excluded technical or aborted attempts; it does not provide full traces for them. Each released record contains aligned request timing and terminal status, control targets, lifecycle events, and resource/cost primitives, with prompts and machine identifiers removed. The standard-library loader and examples recompute feasibility-direction agreement and scenario-cost MAE. The worked screening example also returns mismatched request IDs and reports false accepts/rejects with their physical denominators; `python -B verify.py` checks the complete corpus after extraction. This release supports analysis of the recorded deployments, not unseen-workload validation or rerunning model inference. The later 14B-30, Azure, and Vidur-backed retrospective predictions are supplied separately with the paper and are not included in these counts. The corpus's `SCHEMA.md` specifies the projection and accounting boundaries.

The remaining `scripts/`, `docs/`, and `artifacts/` paths in this supplement identify the separately retained full experiment archive. Its `scripts/maxopt_v3_closeout/reverify.py` rebuilds the 36 acceptance reports, and `scripts/maxopt_v3_closeout/analyze.py` generates their tables; both require archived runs outside the compact source release. A recovery addendum restores the root environment snapshot from identical, hash-bound per-run copies and provides a newly generated archive integrity index. The original checksum sidecar remains unavailable; the new index is not presented as the historical original.

[1] `https://anonymous.4open.science/r/FidelityLoop-144B/`

Table 1. Study names and original archive IDs. Each retains its own runtime, workload, prediction freeze, and analysis denominator.

Paper name	Archive ID
Held-out fidelity	V3
Mixed-policy	Bridge
Same-runtime comparison	E1
Policy deployment	E2
Checkpoint screening	E3
Serving-engine transfer	E4
Arrival transfer	E5
14B model transfer	E6
External validity	V4
Controllers / Llama / Refit	W1 / W2 / W3
Azure full / thinned	W4 / W4-L

- Full-capacity feasibility: nine all2 reports and G-F in `decisions.json`.
- Model accuracy: `prediction_groups.csv`, `common_support.csv`, and `route_confusion.csv`; M1, M1b, and M2 in `decisions.json`.
- H's negative result: 15 records in `misses.csv`; guard risk-active ticks and IDs in `guard_events.csv`.
- Guard feasibility and cost: nine guard runs in `run_metrics.csv`, with repeat-block comparisons in `paired_costs.csv`.
- Production quality, physical energy, and provider billing: unsupported by this dataset and excluded from the claims.

A.1 Display names and archive identities

Reader-facing names describe each study's role and each replay model's timing assumptions. Original archive IDs remain unchanged in released records and filenames; `DISPLAY_NAMES.json` gives the machine-readable mapping. The study labels group related questions, not a pooled execution cohort.

The replay names map as follows: Share (O), Lookup-69 (C), Lookup-30 (C30), Local (E), Legacy-refit (V), Local-refit (V'), Lookup-SG (C_{sg}), 14B-Hist (S14), 14B-Local (L14), and 14B-30 (T30-14). Earlier-local denotes the Azure

Table 2. Stable checkpoint identities across studies. Original labels and training coordinates remain in the corresponding evidence tables.

Paper name	Original display label
PPO-1 / PPO-2	C-1 / C-2
PPO-3 / PPO-4 / PPO-5	C30-1 / C30-2 / C30-3
PPO-6 / PPO-7 / PPO-8	N1 / N2 / N3
PPO-9 / PPO-10 / PPO-11	N4 / N5 / N6
PPO-12	Bridge PPO-C

Table 3. Implementation footprint in code lines. PPO policy code includes 136 lines of reused network implementation. Shared rows are separate from the per-policy counts; backend and training dependencies are itemized in the audit.

Component	Policy	Integration
B-HPA	38	shared
B-PRED	23	shared
B-MMC	29	shared
Deadline guard	142	19
PPO	247	382
Shared baseline support	—	73
Shared runtime control	—	197

archive's E, whose startup is 25.196 s; Local in the held-out study uses the deployment-adapted 33.497 s value. Numeric suffixes in Lookup-69 and Lookup-30 summarize startup seconds; Table 2 maps the distinct learned policies.

The mixed-policy study's PPO-C30 is the same checkpoint as PPO-4 in the deployment study (checkpoint SHA prefix `e03a5695`, tensor SHA prefix `9c6cd1c2`). PPO-12 is distinct (`c77d492d`, `b38f1732`). Both mixed-policy PPO arms control capacity without the guard. Shared checkpoint identity does not merge their runtime groups or physical outcomes.

A.2 Implementation footprint

Table 3 separates policy logic from integration code in the frozen execution sources. We count physical Python lines carrying code, excluding blank lines, comments, and docstrings; shared components are counted once. The audit records file hashes, symbol ranges, and supporting modules, including the inherited PPO training code. These counts describe implementation size, not historical porting effort.

`IMPLEMENTATION_FOOTPRINT.json` binds these counts to the frozen sources. The source release provides the counting script and audited source snapshots in `artifact/implementation_footprint/`.

A.3 External service model in the validation loop

The Vidur integration separates service prediction from the outer control loop. Vidur (Agrawal et al., 2024) supplies its native Sarathi scheduler and token execution events; FidelityLoop supplies policy observations, tick-based admission, lifecycle transitions, and deadline/resource accounting. This allows the same policy and ledger to evaluate an independently profiled service model.

Service validation. New Qwen2.5-7B-Instruct/A100 80GB operator profiles are paired with physical vLLM 0.9.0 service measurements (FP16, TP=PP=1). Six input/output lengths— 128/128, 256/128, 512/128, 1024/128, 2048/128, and 256/256—at concurrency one and four, with three repeats, yield 36 groups and 90 requests; two warmup groups are excluded. Historical prompts supply fixed-length prefixes, with one repeated prefix in the four-request 2048-token group. Output lengths are fixed and prefix caching is disabled.

The profile contains 323 MLP and 761 attention measurements from pinned Sarathi/FlashInfer on PyTorch 2.3; physical service uses vLLM V1/Flash Attention on PyTorch 2.7. Operator profiling precedes service measurement. CPU prediction uses 100-tree, depth-eight random forests (seed 42), Sarathi batch cap four and chunk size 16,384. Inputs contain token budgets and admission offsets, not completion times. CPU overhead is not modeled and no correction is fitted to validation latencies.

Request-latency APE is 18.36% at the median and 19.74% at P90; group makespan APE is 18.36% and 19.66%. The worst length-by-concurrency median makespan APE is 19.80%. These meet the fixed engineering gates of 20%/35% median/P90 for both metrics and 50% worst-condition median. No condition exceeds the registered 1.5 maximum/minimum repeat-makespan ratio. All 36 groups underestimate makespan (10.88–26.40%); the experiment does not isolate runtime-stack effects from omitted overhead.

Policy and lifecycle evaluation. The same profiled service model replaces Local's lookup in 12 CPU rollouts: all1, all2, H, and H+guard on the original three windows. Each deterministic prediction is compared with all three existing physical repeats. This is a retrospective comparison without outcome fitting. Local's startup/shutdown parameters (33.497/2.224 s), the guard's original lookup and safety estimates, and the synthetic API rules stay fixed. Thus lifecycle timing comes from FidelityLoop's frozen deployment model, not from Vidur's operator profiles.

Both instances begin ready. Completions advance in continuous time; each one-second tick releases arrivals, evaluates the policy, and fills free local slots (four per active instance).

Table 4. Vidur-backed dynamic-policy comparison. Offline on-time counts use 120 arrivals; physical ranges span three repeats. Resource APE is the median over those repeats; target agreement is the percentage of decision ticks with equal target counts, reported as a range.

Policy	Window	CPU on time	Physical on time	Resource APE (%)	Target agree (%)
H+guard	burst	120	120	0.77	67.9–99.6
H+guard	recovery	120	120	14.74	72.1–99.3
H+guard	steady	120	120	2.47	33.0–35.5
H	burst	120	120	0.91	69.6–99.6
H	recovery	116	113–116	0.84	90.9–99.3
H	steady	120	120	2.47	35.3–35.5

Draining instances finish accepted work before shutdown; no new work enters them. Occupation integrates all non-off states over [0,2100] s, excluding setup and cleanup. Unfinished requests remain in the deadline denominator. Intermediate input lengths and continuous mixed prefill/decode batches extend beyond the isolated service-screen cases. All 712,614 native batch queries remain within the measured operator domain, including 2,340 mixed batches; this domain check does not assert per-shape service accuracy.

Table 4 reproduces the recovery deadline contrast: H predicts 116/120 on-time offline jobs versus physical counts of 116, 113, and 116; H+guard gives 120/120 in both prediction and execution. All online requests meet deadlines in both. The six dynamic conditions have median resource APE of 0.77–2.47%, except guard recovery at 14.74%. Aggregate accuracy does not imply a matching control trajectory: steady-window target agreement is 33.0–35.5%, and predicted API routing is 755 requests versus physical medians of 711 for H and 714 for H+guard. Fixed all1/all2 median resource APE is at most 0.02%, largely because capacity is fixed over the same observation window. These are descriptive comparisons, not a newly applied policy-accuracy gate.

Reproduction. `artifact/vidur_service_screen/` supplies operator profiles, request timings, predictions, and service-score verification. `artifact/vidur_control/` supplies reduced per-request results, lifecycle intervals, control targets, and a standard-library verifier for all 12 predictions and 36 physical comparisons. These files accompany the paper; the release also supplies a CPU runner for the fixed predictions. They do not add physical runs to the 197-run paired corpus. Interface checks also preserve the earlier public-Llama native-output example in `artifact/vidur_cpu/`. Across the Qwen interface checks, native completion times match the accepted service-screen predictions exactly, and independent ledgers reproduce the saved occupation and deadline counts.

A.4 Per-repeat latency record

Table 5 preserves all three online tail statistics for every formal run, from the same `run_metrics.csv` records used in the main-text summary.

Table 5. Per-repeat online p95, p99, and maximum latency in seconds.

Policy	window	repeat	p95	p99	max
all2	steady	1	1.598311	1.613852	1.934303
all2	steady	2	1.600293	1.613611	2.053080
all2	steady	3	1.598100	1.612091	2.338287
all2	recovery	1	1.573470	1.581393	1.587289
all2	recovery	2	1.577489	1.584092	1.587016
all2	recovery	3	1.572175	1.582705	1.583298
all2	burst-offline	1	1.571978	1.582704	1.673667
all2	burst-offline	2	1.573614	1.594019	1.678853
all2	burst-offline	3	1.567986	1.583607	1.675901
all1	steady	1	2.611935	2.632597	3.617594
all1	steady	2	2.606561	2.622125	3.621537
all1	steady	3	2.615337	2.631706	3.626181
all1	recovery	1	1.611564	1.626285	2.564877
all1	recovery	2	1.616560	1.683677	2.581092
all1	recovery	3	1.620432	1.686686	2.577571
all1	burst-offline	1	1.647122	1.704174	1.749571
all1	burst-offline	2	1.612728	1.632177	1.732826
all1	burst-offline	3	1.605751	1.637405	1.729437
H	steady	1	17.001106	19.002347	21.001175
H	steady	2	17.000396	19.001483	21.001591
H	steady	3	17.001118	19.001507	21.002980
H	recovery	1	15.002311	15.008397	15.011078
H	recovery	2	15.002046	15.009679	15.014929
H	recovery	3	15.002045	15.008747	15.012199
H	burst-offline	1	15.003550	15.009867	15.011047
H	burst-offline	2	15.004760	15.046479	15.116333
H	burst-offline	3	15.002258	15.008656	15.009492
H+guard	steady	1	17.001446	19.002137	21.005507
H+guard	steady	2	17.001417	19.001712	21.005507
H+guard	steady	3	17.001367	19.001675	21.004582
H+guard	recovery	1	15.002007	15.008547	15.009813
H+guard	recovery	2	15.002468	15.008545	15.046810
H+guard	recovery	3	15.002523	15.008521	15.031079
H+guard	burst-offline	1	15.001992	15.003827	15.009511
H+guard	burst-offline	2	15.002109	15.005205	15.011248
H+guard	burst-offline	3	15.002959	15.024269	15.046434

A.5 Evidence provenance and scope

The workload table is generated from `workload_manifest.json`, whose source chain records the 1,404,294-row BurstGPT-derived input, the parent manifest, the forward pool selection, and the fixed thinning threshold. The selected windows have exact prompt overlap counts of 60, 34, and 30 with their old parent windows in steady, recovery, and burst-offline, respectively; this is disclosed as an identity limitation rather than treated as session independence. The source has no session field, and the offline stream is fixed synthetic

Table 6. Separate 38-cell mixed-policy study. Entries show recovery/steady P1+ counts, followed by operating cost in the same order.

Policy	P1+ R	P1+ S	occupied R/S	cost R/S
all2 (capacity anchor)	1/1	1/1	4200/4200	4.200/4.200
H-d0	3/3	0/3	1625/1778	1.752/2.135
H-d0+guard	3/3	0/3	1623/1975	1.752/2.314
H-d1	0/3	3/3	1547/1841	1.709/2.208
H-d1+guard	0/3	3/3	1707/1837	1.870/2.226
PPO-4	1/3	3/3	1387/1991	1.626/2.075
PPO-12	0/3	0/3	1184/1218	2.547/2.935

Table 7. Raw-event guard audit for the 12 guarded cells in the separate mixed-policy study campaign.

Down	window	risk runs	raises/run	misses/360	$P1+$ pass
d0	recovery	0/3	0	0/360	3/3
d0	steady	3/3	2.0	9/360	0/3
d1	recovery	3/3	2.0	7/360	0/3
d1	steady	0/3	0	0/360	3/3

pressure.

The price table is a revaluation of the same event streams over 27 price cells; it introduces no new rollout or physical run. The guard table counts recorded risk-active ticks, distinguishing target raises from ticks that merely maintain a latched reservation. Two operator-aborted attempts remain in the raw history and are excluded from the 36 formal runs; the 15 H recovery misses remain in the formal evidence and are not replaced by guarded outcomes.

A.6 Mixed-policy evidence and guard audit

The mixed-policy study contains 38 locked cells: an all2 capacity anchor, two hysteresis down-threshold settings with and without the guard, and two frozen PPO policies, each evaluated on recovery and steady windows. It is historical evidence from a different runtime group, not an extension of the deployment cohort. The compact policy summary is shown below before the raw-event audit.

The mixed-policy guard audit joins each recorded risk ID to its request terminal and absolute deadline; it does not rerun the policy or form a counterfactual. Each cell below aggregates three guarded repeats. A risk-active run has at least one nonempty risk-ID set; target raises are averaged over risk-active runs; deadline misses count late or unfinished offline requests across the three repeats.

B HELD-OUT ACCOUNTING AND GATE DETAILS

Completed requests have a completion terminal by the observation cutoff. On-time completion includes equality with the deadline; late completion is strictly after it. Unfinished requests lack a completed terminal at cutoff, and missing or censored requests are not removed from the arrival denominator. G_F requires every all2 online and offline arrival on time. P1 requires 120/120 offline arrivals on time, at least 99% of online arrivals on time, and no unfinished request. P1+ additionally requires every online arrival on time. These definitions follow `scripts/maxopt_v3/acceptance.py`; the extension's strict qualification uses the same timely/unfinished distinction.

B.1 Replay model specification

The main text lists the five frozen replay models and controlled contrasts. This section gives their construction and provenance. The deployment study reuses the frozen Lookup-69/Lookup-30 pair.

Service construction. For input length i, output budget o, and local concurrency c, Share assigns service duration $c(i/1024 + o/128)$ seconds, using assumed rates of 1024 input and 128 output tokens/s. Lookup-69 replaces this formula with the median measured wall time per homogeneous input/output/concurrency cell. The replay interpolates linearly in input length between 128, 512, and 2048 tokens, and in concurrency between one and four, with output budgets restricted to 32, 128, or 256 tokens. A dedicated 256-input/256-output, concurrency-four measurement gives 3.098 s. Each running request accumulates progress at the reciprocal of its current-concurrency service estimate; this is a mixed-workload approximation, not token-level vLLM simulation. Lookup-30, Local, and Legacy-refit share this lookup unchanged.

Lifecycle construction. Share's 30 s startup is the original assumed constant. Lookup-69's 69.219 s proxy is the median of 18 launch-to-ready observations collected during two-instance initialization in nine development runs. The shared historical 7.059 s shutdown proxy comes from nine joint-cleanup durations; the 69.363 s setup proxy is the median of nine setup-barrier durations. Lookup-30 restores the 30 s assumption while holding Lookup-69's other fields fixed; it is not a new fitted startup estimate.

The earlier empirical lifecycle model separated window restarts from setup and normal scale-in from terminal cleanup. Medians within each run, followed by a median across runs, gave startup/shutdown values of 25.196/1.937 s. Legacy-refit searched the nine combina-

tions of startup multipliers $\{0.9, 1.0, 1.1\}$ and shutdown multipliers $\{0.8, 1.0, 1.2\}$ around that center, clipped below by the corresponding observed minima. It minimized equal-window L on the earlier development runs; leave-one-window-out and leave-one-repeat-out checks preceded the final fit. The frozen Legacy-refit uses 25.196/2.325 s and is distinct from the later Local-refit model.

Before the held-out study, Local alone was adapted to the A100 deployment: medians from six development restarts gave 33.497/2.224 s, followed by six confirmation restarts. The service lookup, other models, and selected policy were unchanged; no held-out outcomes entered this adaptation. Thus Legacy-refit–Local compares the historical refit with the deployment-adapted model, rather than isolating a fitting-method effect. Windows begin with both instances ready; the inherited setup proxy is not fitted through the window loss. The accompanying `REPLAY_MODEL_SPECIFICATION.json` retains the full lookup, unrounded parameters, and source hashes.

Local's equal-window target disagreement is 26.77%, against the registered 25% diagnostic expectation. Predicted and observed switch counts match except for one burst-offline repeat (22 predicted, 24 observed).

B.2 Guard implementation map

The frozen `scripts/maxopt_v3/policy.py` implements the following causal projection and state update. The lookup is a conservative estimate, not a proved upper bound, and infeasible work is still reported rather than guaranteed.

- `validate_observation` (lines 20–47) accepts only released queues, device states, state ages, and running-job dispatch times. Future arrivals, true readiness, and completion times are not policy inputs.
- `duration/remaining/delay` (70–90) uses concurrency four. A running job receives its estimated residual if positive, otherwise a full-service fallback. Active/draining devices have zero added delay; an off device needs the frozen safe startup time. A starting device uses the remaining safe-start interval, then the maximum-start interval (both floored at one second), then restarts the positive safe-start estimate if both are exceeded. A stopping device adds remaining shutdown, or a full shutdown fallback, before startup.
- `projected` (92–126) chooses the k devices with least lifecycle delay, with device-ID ties, and builds four slots each. Offline jobs are sorted by deadline and ID. Without reservation, current online backlog precedes offline work. With reservation and online backlog, offline jobs are serialized from the earliest running offline slot, or the earliest slot if none is running. Without online backlog they use all slots. No future arrivals are projected.
- `decide` (128–158) first obtains H's target b and intersects latched risk IDs with pending offline IDs. It adds IDs with projected finish $F_j(\max(1, b), r) + m + 1 \geq d_j$, where $m = 5$ s. With active risk and scaling enabled, it tests $k \in \{1, 2\}$ using $F_j(k, r) + m \leq d_j$ for *every* pending offline job. The target is $\max(b, \min k)$ when feasible; otherwise it is two and `infeasible` is set. A raise bypasses cooldown and updates H's target and last-change time. Safe re-estimation alone does not clear a latched ID; dispatch or removal from the pending queue does. Reservation is enabled only while risk is latched and the reservation option is enabled.
- `dispatch_kind` (160–172) never preempts running work. With both queues nonempty and a reservation active, an already-running offline job leaves free slots online-first. Otherwise online takes the next slot only when its head request's estimated slack is at most m and no larger than the offline head's slack. With no online queue, offline dispatch is unrestricted by the single-slot reservation; without an active reservation, online has priority.

Table 8. Accepted held-out cost ledger. Costs are means over three repeats; the final two columns are savings percentages.

Window	all1	H+guard	window	deployment
steady	2.1029	1.6411	21.96	21.41
recovery	2.1029	1.0895	48.19	37.82
burst-offline	2.1029	0.8815	58.08	57.34

Table 9. Gate interpretation for the held-out study; M3 is diagnostic.

Gate	observed result	interpretation
G-F	all2: 9/9	full-capacity feasibility
M1	Local: 87.50% reduction Legacy-refit: 34.24%	occupied-resource gate
M1b	L reduction Share fails;	environment caveat
M2	lookup-based models pass	matched-service fidelity
M3	Local: 26.77%	diagnostic, no veto
P1+	H+guard: 9/9	online/offline feasibility
P2	mean: 42.74%	window-cost gate

B.3 API-delay sensitivity in CPU replay

We replay the four frozen rule policies (all1, all2, H, and H+guard) on the three original windows under model Local, varying only the fixed API service delay over $\{1, 2, 5, 10, 20, 40, 60\}$ s. API capacity remains eight concurrent requests, with the same 10 s routing wait, prices,

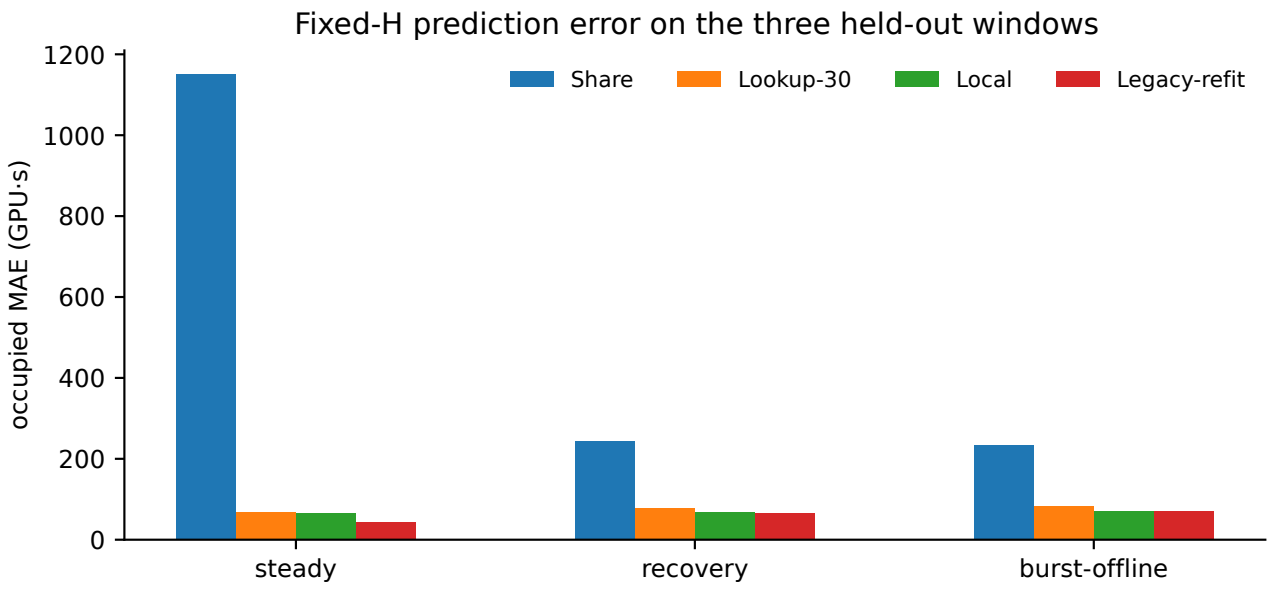


Figure 1. Occupied prediction error on fixed H. Lookup-30, Local, and Legacy-refit have lower error than Share on all three windows; the main-text table also includes Lookup-69.

and policy parameters. All 12 five-second baselines reproduce the frozen predictions, yielding 84 deterministic CPU scenarios in total.

At the tested delays of 1, 2, 5, and 10 s, both H+guard and all1 satisfy P1+ in all three windows. Their equal-window cost savings are 45.07%, 45.07%, 45.08%, and 43.76%, respectively. At 20 s, H+guard misses online deadlines in recovery and steady; at 40 and 60 s, online misses occur in all three guarded windows. Longer API service occupies the finite API slots and changes subsequent queueing, routing, and GPU control. These results test sensitivity to the timer assumption; they are CPU predictions, not additional physical runs or an empirical calibration of a commercial API.

C External-validity evidence ledger

The external-validity source records supplement, rather than replace, the 36 frozen held-out runs. Table 10 distinguishes new execution, historical reuse, and gate roles. All 93 accepted identities are included once; the full-strength Azure load study's 18 gate-stopped policy cells are not counted as executions. The thinned-Azure study's p075 and p050 gate batches are qualification evidence, not additional policy repeats.

Strict qualification requires at least 99% online on-time completion, all offline jobs completed by their deadlines, and every expected request completed without censoring or missing work. Offline counts use each input's actual expected denominator; formal comparison windows here contain 120 jobs, whereas development windows may contain a different count or none.

The generated external-validity rows are bound to the CPU closeout's `paper_data/TABLE_MANIFEST.json`, `run_metrics.csv`, `window_summary.csv`, `equal_window_summary.csv`, and `w4l_pairs.csv`. The source root is `artifacts/max_optimization_v4_20260923/cpu_closeout_20260927/`. Repeated requests in these CSVs are evidence about a run, not independent replications. The held-out study's P2 uses the equal average of within-window ratios; the thinned-Azure study headline instead uses a ratio of equal-window means. The two statistics are reported under their own definitions and are not numerically pooled.

C.1 Baseline observation and selection contract

B-HPA updates every 60 seconds using the preceding queue observations; its registered grid is $q_{\text{target}} \in \{4, 8, 16\}$ and stabilization $\in \{120, 300, 600\}$ seconds. B-PRED uses Holt level/trend updates with $\alpha \in \{0.2, 0.4, 0.6\}$, $\beta \in \{0.1, 0.3, 0.5\}$, horizon $\in \{60, 120, 300\}$ seconds, and margin $\in \{0, 1, 2\}$ (81 candidates). B-MMC's utilization ceiling is drawn from $\{0.70, 0.80, 0.85, 0.90\}$. Its service lookup is shared with Local to isolate the target rule; this does not provide future information or make it an oracle. Development ranking places feasibility before equal-window cost; the preregistered center is retained if a family has

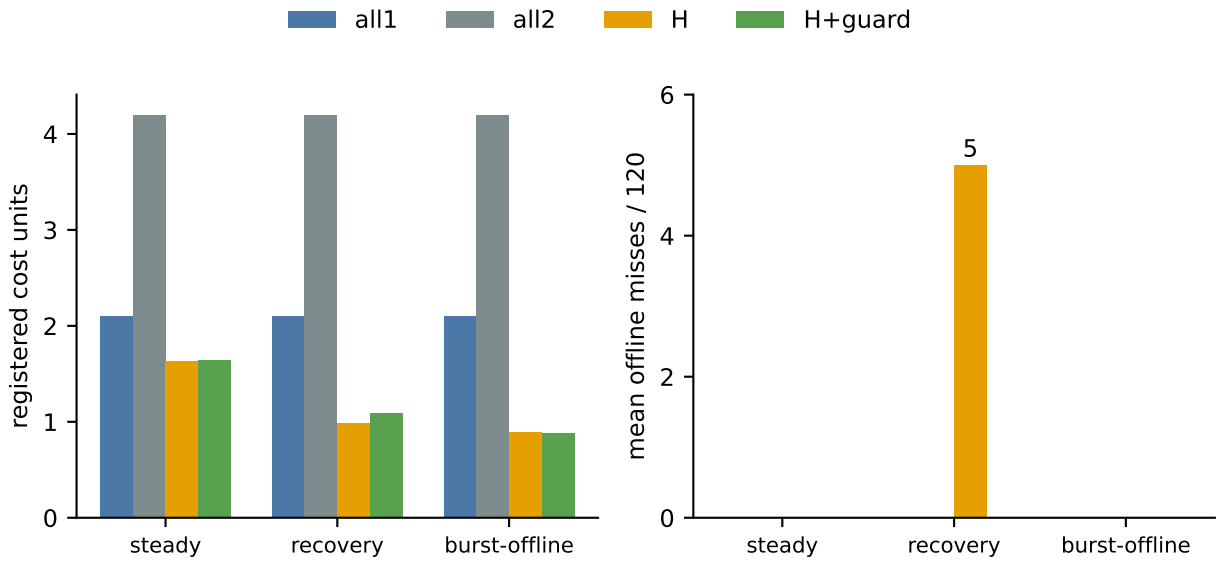


Figure 2. Window cost and offline deadline misses, averaged over three repeats per arm/window. H averages five recovery misses (15 total); all other bars are zero. The guarded arm has no misses while reducing scenario cost relative to all1.

Table 10. External-validity coverage and strict qualification. The thinned-Azure study p050 fresh contains all four policies; its 15/24 total must not replace their per-policy counts.

Workstream	fresh	reused	identities	strict	role
Controllers	27	0	27	27/27	comparison
Llama	18	6	24	21/24	comparison
Azure full load	6	0	6	0/6	gate
Azure thinned p075 gate	6	0	6	0/6	gate
Azure thinned p050 gate	6	0	6	6/6	gate
Azure thinned p050 fresh	24	0	24	15/24	comparison
Total	87	6	93	—	mixed roles

no qualifying candidate. None of the three formal arms includes the guard.

All 9/81/4 candidates, respectively, failed development qualification. The predeclared fallback therefore fixes B-HPA at queue target 8 and stabilization 300 seconds, B-PRED at $\alpha = 0.4$, $\beta = 0.3$, horizon 120 seconds and margin 1, and B-MMC at $\rho_{\max} = 0.80$. The original development feasibility field incorrectly rejected windows containing zero offline jobs. Recalculation with each window's actual offline denominator still leaves all 94 candidates ineligible: each independently violates the 99% online criterion. Thus the fallback is unchanged, but the original field is not treated as authoritative. The subsequent fixed-center validation repairs the preceding-60-second interval to $[t - 60, t)$ and checks the same three centers. It performs no grid search and reads no formal or locked-test outcome; it is semantic validation, not retuning.

Table 11. Alternative-controller online end-to-end p99 latency (seconds): minimum–maximum of the three completed-request per-run quantiles. These are repeat ranges, not confidence intervals or pooled request quantiles.

Policy	steady	recovery	burst
B-HPA	19.00–19.00	15.00–15.00	1.66–1.81
B-PRED	1.61–1.68	1.58–1.59	1.60–2.70
B-MMC	19.00–19.00	15.00–15.00	1.58–2.01

The runtime continuation is a comparability limitation, not an exclusion rule. Historical alternative-controller cells 001/002 and continuation r14 remain in the nine-per-arm denominators. Comparisons with the held-out study are descriptive; their repeat indices do not establish randomized pairing, and request-level resampling would not correct the confound.

Table 12. Sequential all2 qualification. Front/back list timely offline jobs out of 120 for each of three repeats. Gates are excluded from the fresh policy comparison.

Intensity	front	back	strict	action
1.00	35/34/35	37/37/37	0/6	stop
0.75	113/113/113	79/79/79	0/6	lower load
0.50	120/120/120	120/120/120	6/6	lock; fresh 24

C.2 Failure records that constrain the claims

The Llama study H recovery cells 042/043 each contain 116 completed jobs out of the original 120; four unfinished jobs per cell remain in the denominator. Guard recovery cell 049/r2 completes 120 but has only 119 timely jobs. These facts explain why 24/24 technical validity, 23/24 historical scientific acceptance, and 21/24 strict qualification are not contradictory. The Llama guard-feasibility criterion is not established by a technical or historical aggregate pass.

For the Azure load study, the full-strength all2 gate stops the policy matrix after six scientific failures. The p075 amendment gate also fails, despite eventual completion of all offline work. The first feasible p050 gate permits, but does not substitute for, 24 fresh executions. In those fresh records H front/r1 and front/r3 each leave two offline jobs unfinished; H back/r2 leaves four. all1's six failures remain part of the official negative aggregate. No scientific failure is relabeled as an operator abort, deleted, or replaced by a guarded result.

C.3 Source and intensity provenance

The Azure input is the conversation CSV at immutable repository commit `207bed67dd10090b28ad4f745b2cfd41a11aace4`, with SHA-256 `2f1e5b666d4e3055fdbba98598ce2ec307767b9064e03e2fa46676dbcc7d0bf8`. Its three columns are timestamp, context tokens, and generated tokens; only timestamps construct the replay arrivals. The code trace is archived but not substituted. The thinned-Azure integer-SHA rule preserves the parent source row and timestamp of every retained online arrival and keeps the offline stream unchanged. Thus p050 means a locked online inclusion intensity, not half the offline work or a rescaled time axis. Neither p050 nor the unused optional p025 is a new independent sample of Azure production workloads.

Independent CPU checks regenerate inputs and request metrics from raw records, and bind worker events to request identity and lifecycle generations. The supplemental provenance package closes the three missing original-source links; its isolated replay test suite passes 107 tests. This is verification of the available artifact, not new GPU evidence. The original held-out convenience-snapshot warning remains historical and is not silently rewritten.

Table 13. Thinned-Azure study 0.50 fresh comparison. Equal-window means preserve all three repeats in each window. Only all2 and H+guard qualify strictly in all six runs. Costs are scenario USD, not measured provider bills.

Policy	strict	occupied	window	deployment
all2	6/6	4200.0	4.2000	4.3204
all1	0/6	2101.9	3.5248	3.6093
H	3/6	2958.9	3.0428	3.1186
H+guard	6/6	2992.4	3.0675	3.1366

Table 14. Thinned-Azure study p050 fresh equal-window segment means. Costs are scenario USD; occupied time is GPU seconds. The final 600 seconds supply about 81.96% of guard's absolute whole-window saving against all2.

Policy	seconds	occupied	cost	API cost
all2	0–1500	3000.0	3.0000	0.0000
all2	1500–1800	600.0	0.6000	0.0000
all2	1800–2100	600.0	0.6000	0.0000
all1	0–1500	1501.9	2.8609	0.5779
all1	1500–1800	300.0	0.3639	0.0039
all1	1800–2100	300.0	0.3000	0.0000
H	0–1500	2738.8	2.8034	0.0511
H	1500–1800	181.2	0.1864	0.0000
H	1800–2100	38.8	0.0530	0.0000
H+guard	0–1500	2728.1	2.7957	0.0536
H+guard	1500–1800	202.4	0.2081	0.0000
H+guard	1800–2100	61.9	0.0638	0.0000

C.4 Time-segment cost accounting

The first interval is the nominal 1,500-second source-arrival phase, not the whole replay. The next interval still contains 20 offline arrivals per window, and the back window's closed endpoint retains an online arrival at 1500 seconds. The $[1800, 2100)$ interval extends observation through the remaining deadline horizon. These intervals are accounting partitions, not claims that every non-source second is empty or that the entire last 600 seconds represent a production steady state.

D CPU MODEL EVIDENCE

D.1 Llama: within-model gates and common support

The Llama analysis aligns 48 O_x/E_x predictions with 24 physical sources. It preserves the six historical all2 identities exactly once. Both models are derived from development calibration only: O_x uses a cell median and E_x a cell maximum. The simulator consumes the two-GPU aggregate

Table 15. Llama fixed-H model error; O_x and E_x have the same values at the shown precision. Each window first averages its two physical repeats.

Window	L (both models)	occupied MAE
steady	0.01170749	73.09045
recovery	0.00059876	1.44829
burst-offline	0.00032530	0.89862
equal-window	0.00421052	25.14579

lookup, not the per-GPU metadata. Both have startup/setup 29.213463 seconds and shutdown 1.917728 seconds. Actual guarded execution uses its fixed E_x plan; each counterfactual predictor uses its corresponding model with the same policy and safety settings. No formal outcomes fit these service or lifecycle values.

For horizon T, two devices, and N total arrivals, the registered loss is

$$L = \frac{0.5|\Delta G_{\text{occupied}}| + 0.25|\Delta G_{\text{active}}|}{2T} + \frac{0.25|\Delta Q|}{TN}, \quad (1)$$

where Δ is prediction minus observation, $T = 2100$ s for formal windows, and N includes all online and offline arrivals. Q is the integrated released-but-undispatched queue in request seconds. Unfinished jobs remain in N and contribute queue time while queued up to cutoff. Losses are averaged over repeats within each window and then equally across windows, not over pooled requests or matched-local support. The bounded-refit development horizon is 900 s. This prediction-error objective is distinct from monetary scenario cost. The Llama main gate is L reduction $\geq 20\%$ on H together with per-window occupied MAE $\leq O_x + 21$ GPU seconds. The inherited occupied-improvement threshold is a separate diagnostic, not an alternative route to a pass.

M2 uses the intersection of actual-GPU, O_x, and E_x completed-local IDs. Pairwise support is retained separately even though it coincides with common support in these records. Fixed and dynamic M2 criteria pass, with minimum common support 206 and maximum per-run common MAE 0.02963983 seconds. This is not a whole-stream fidelity claim: equal-window coverage over all arrivals is 100% for all1/all2, 51.2417% for H, and 58.8975% for guard. In H's two steady repeats, only 454/457 requests are common-local, respectively, covering 29.0281%/29.2199% of arrivals and 53.4747%/54.0189% of actual-local completions. Targets disagree with the actual controller at 1401/2100 and 1397/2100 ticks despite matching switch counts of 28. Equal-window target disagreement is 22.2063% for H and 0.6190% for guard.

The zero main-metric improvement is not accidental model duplication: 14,324 requests have different O_x/E_x predicted completion times. Those small service changes do not sufficiently change the registered resource and queue metrics. The r2 gate-only repair restores the preregistered safety conjunction; the 48 predictions and numeric tables remain byte-identical to r1. Independent recalculation and boundary tests accept this implementation while retaining the scientific negative result. Evidence resides in `w2/decisions.json`, `w2/prediction_errors.csv`, `w2/route_confusion.csv`, and `w2/target_comparison.csv` under the CPU closeout root stated above.

Table 16. Bounded-refit fixed-H replay. Each window averages three physical repeats; occupied MAE is in GPU seconds. Steady L worsens. All occupied safety lines pass, but the equal-window improvement of 5.02% misses the 10% gate.

Window	Local L	Local-refit L	Local occupied	Local-refit occupied
steady	0.012390	0.014516	66.34	70.67
recovery	0.012889	0.010865	66.95	66.95
burst-offline	0.015303	0.013164	70.55	70.55

D.2 Bounded lifecycle refit

The bounded-refit development bundle contains a preformal drift record and two historical Qwen engineering canaries (sparse and recovery), each with one observation. Their model bytes, engine configuration, and platform match the frozen formal deployment. The canaries contain 41 and 1440 ordinary completed GPU service pairs, respectively; health probes are excluded. Their request IDs are disjoint from formal IDs. These dynamic records are not independent fixed-concurrency experiments or additional formal repeats.

A nine-cell grid varies startup by $0.9/1.0/1.1$ and shutdown by $0.8/1.0/1.2$ around Local, subject to preformal observed floors of 24.958327 and 1.897256 seconds. The development loss uses the same dimensionless definition above at its 900-second horizon, then averages the two windows equally. The selected Local-refit has startup 30.146928 seconds and shutdown 2.224161 seconds; only startup differs numerically from Local. Setup, the service lookup, policy, guard model Local, and all guard safety settings remain frozen. Prepare, fit, and evaluate locks are ordered and hash-bound; formal target/event/occupied values do not enter selection. Nevertheless, the prior held-out aggregate outcomes were already known, so the procedure cannot be described as a new blind study.

All 36 accepted formal identities enter evaluation. Twelve unique deterministic replays per model are compared with the physical repeats, producing 72 model-by-run error records; these are not 72 new executions. Local's summary, requests, and events exactly reproduce all 12 frozen held-

out study prediction groups. The primary fixed-H equal-window loss is 0.0135273755 for Local and 0.0128485693 for Local-refit, a reduction of 5.018019%. The conjunctive improvement-and-safety gate therefore fails despite three passing Local + 21 safety checks. Guard's 4.073089% reduction is descriptive and cannot replace the primary H result; all1/all2 are unchanged.

A technical correction in evaluation maps accepted short window names to their full locked identifiers. It precedes prediction output and outcome comparison, leaves the fitting grid/loss/tie-break and frozen model unchanged, and is retained in `w3/evaluation_technical_amendment.json`. Independent review reproduces the raw ledgers, selection, loss, and representative complete simulator objects. The accepted conclusion remains negative with two-canary, retrospective-knowledge, and lifecycle-only scope limits. Evidence includes `w3/prepare/analysis_plan.json`, `w3/fit/frozen_model.json`, and `w3/evaluate/result.json`; no new GPU run is introduced.

E SAME-RUNTIME BASELINE COMPARISON

The same-runtime comparison compares frozen H+guard and B-HPA on the previously observed `locked_test_steady_v3` and `locked_test_recovery_v3` inputs without revising the held-out study/external-validity locks. All 110 formal lifecycle generations share the two-A100 Qwen2.5-7B-Instruct FP16 runtime, model closure, service lookup, health probe, synthetic API, and accounting contract. Both instances start physically ready with empty queues; adapter targets remain B-HPA=0 and H+guard=2. These targets and the intrinsic dispatch rules belong to the complete policy treatments, so the same-runtime comparison does not isolate the causal contribution of the guard module.

E.1 Identities, feasibility, and retained smoke outcomes

The formal matrix contains one all2 capacity run per window and three fresh repeats of each comparison policy per window: 2 capacity plus 12 comparison identities. Each steady run contains 1,444 online and 120 offline arrivals; each recovery run contains 334 online and 120 offline arrivals. Independent raw recomputation finds all 14,126 formal arrivals completed on time, with zero late, failed, or unfinished requests. Both capacity runs pass G_F and every comparison run passes P1/P1+. Table 17 retains all 14 identities; the capacity runs are qualification evidence, not extra B-HPA or guard repeats.

The 17 entries include three 90-second nonformal smokes. B-HPA a1 is `TECHNICAL_INVALID`: its lifecycle callback rejected ready after forced stopping, leaving four offline arrivals unfinished. After repair, a2 records 68/68 online completions on time but 0/4 offline completions at cutoff; its `NEGATIVE_RESULT` and false gate remain intact. The wrapper's `PASS` denotes technical integration only. Guard's smoke completes all 68 online and four offline arrivals on time. None enters the formal comparison. The raw `formal_accepted=false` field is not overwritten; scientific qualification is independently derived from request and lifecycle records.

E.2 Resource use and cost boundary

For window w, let $\overline{C}_{g,w}$ and $\overline{C}_{h,w}$ be the arithmetic means across the three H+guard and B-HPA repeats. The same-runtime comparison reports $R_w = 1 - \overline{C}_{g,w}/\overline{C}_{h,w}$ only after both arms qualify in all three runs, then $R = (R_{\text{steady}} + R_{\text{recovery}})/2$. This is the equal average of within-window ratios, not a pooled ratio or the average of six per-pair percentages. The fixed repeat blocks support descriptive paired differences, not confidence intervals or population-level significance. Window scenario-cost reductions are 32.5975% and 49.3598%, hence 40.9786% equally weighted. Full-deployment reductions are 31.7650% and 47.8629%.

Occupied time spans `launch_issued` through `verified_release`, including starting and stopping, clipped to each phase. Window occupied-time reductions are 50.8839% and 56.1240%; the corresponding mean savings are 1,224.82 and 1,147.44 GPU seconds. This is local resource occupancy, not total compute across the local and external services. Active time is not GPU compute utilization. Guard's zero cleanup cost means that release already completed within the window, not that release or setup is free.

Repricing the fixed ledgers makes the cost boundary explicit. Let ΔC_G, ΔC_A, and ΔC_E denote mean H+guard minus B-HPA GPU, API, and lifecycle-event costs in scenario USD. Their steady values are $(-1.2248, 0.3862, 0.0340)$; recovery gives $(-1.1474, 0.0930, 0.0320)$. With GPU price multiplier x, a common API input/output-budget multiplier y, and fixed lifecycle-event prices, the cost difference is $\Delta C(x, y) = x\Delta C_G + y\Delta C_A + \Delta C_E$. Neither arm incurs missed-job penalties. Since $\Delta C_A > 0$, H+guard costs less when

$$y < y^*(x) = -\frac{x\Delta C_G + \Delta C_E}{\Delta C_A}.$$

At $x = 1$, the API break-even multipliers are 3.08 and 11.99 (3.09 and 12.02 for full deployment). At $y = 1$, window savings require GPU price multipliers above 0.343 and 0.109. Lower local GPU occupancy therefore need not imply lower cost: the boundary depends on external-service prices. This analysis holds routing and lifecycle events fixed, without retuning either policy.

Table 17. All formal runs in the same-runtime comparison. Occupied time is window GPU seconds; costs are scenario USD. API is the percentage of all online arrivals, p99 is each run's completed-online end-to-end quantile in seconds, and S/D counts window starts/shutdowns. Every row completes all online and offline arrivals on time.

Window	Policy	Repeat	Occupied	Window cost	Full cost	API (%)	p99	S/D
steady	all2	gate	4200.0	4.2000	4.2700	0.00	1.616	0/0
steady	B-HPA	1	2407.0	2.4678	2.5389	6.37	19.002	2/3
steady	B-HPA	2	2407.1	2.4696	2.5398	6.58	19.005	2/3
steady	B-HPA	3	2407.2	2.4680	2.5383	6.37	19.003	2/3
steady	H+guard	1	1266.0	1.7611	1.8339	53.25	19.002	13/15
steady	H+guard	2	1140.5	1.6153	1.6842	50.83	19.002	13/15
steady	H+guard	3	1140.3	1.6151	1.6794	50.83	19.002	13/15
recovery	all2	gate	4200.0	4.2000	4.2758	0.00	1.586	0/0
recovery	B-HPA	1	2044.6	2.0715	2.1415	11.68	15.002	1/2
recovery	B-HPA	2	2044.4	2.0713	2.1386	11.68	15.002	1/2
recovery	B-HPA	3	2044.4	2.0713	2.1439	11.68	15.002	1/2
recovery	H+guard	1	1102.2	1.2576	1.3279	60.18	15.008	12/14
recovery	H+guard	2	794.5	0.9447	1.0110	58.98	15.008	11/13
recovery	H+guard	3	794.4	0.9446	1.0104	58.98	15.008	11/13

Table 18. Same-runtime comparison mean phase costs (scenario USD), three repeats per comparison arm/window. Full includes setup, window, and cleanup.

Window	Policy	Setup	Window	Cleanup	Full
steady	B-HPA	0.0675	2.4685	0.0031	2.5390
steady	H+guard	0.0687	1.6638	0.0000	1.7325
recovery	B-HPA	0.0670	2.0714	0.0030	2.1413
recovery	H+guard	0.0675	1.0489	0.0000	1.1164

Table 19. Same-runtime comparison trade-offs. S and D are mean window starts and shutdowns; Q is mean queue request-seconds. p99 ranges span three per-run completed-online quantiles (seconds), not confidence intervals or pooled-request quantiles.

Window	Policy	S	D	Q	p99 range
steady	B-HPA	2.00	3.00	1674.9	19.002–19.005
steady	H+guard	13.00	15.00	12884.7	19.002–19.002
recovery	B-HPA	1.00	2.00	751.1	15.002–15.002
recovery	H+guard	11.33	13.33	9429.1	15.008–15.008

E.3 Routing, lifecycle, and queue trade-offs

The online API fraction uses all online arrivals as denominator, not only local completions. Its mean rises from 6.44% to 51.64% in steady and from 11.68% to 59.38% in recovery. Table 19 retains the additional lifecycle events and queue exposure. Queue exposure sums each request's actual release-to-dispatch time, or release-to-cutoff time if undispatched, in request seconds. It is neither a queue-latency p99 nor a new value of the historical prediction loss L. Tail latency uses completed online requests; the all-arrival feasibility checks above independently rule out hidden formal failures.

The two known inputs and three repeats per cell establish a bounded policy comparison. Greater API use and queue exposure rule out all-dimensional dominance; quality-equivalent API substitution, energy, provider bills, and new-trace generalization remain unmeasured.

E.4 Independent recomputation and artifact links

The accepted table source is `docs/maxopt_v4_r3_p5_20260929/ANALYSIS/P5_RAW_v2/`: `FORMAL_RUNS.csv` retains 14 formal identities, `ATTEMPTS.csv` all attempts, `PAIRS.csv` six repeat blocks, and `AGGREGATES.json` the qualified estimands. `COMPARABILITY.json` binds plans, runtime, and permitted policy differences. In the parent documentation directory, `REVIEW_B.json` and `REVIEW_B_V2.json` bind accepted output hashes; `P6_GENERATE_E1_TABLES.py` and `P6_TABLE_MANIFEST.json` reproduce and bind the displayed rows. The standalone same-runtime package reproduces the resource and price-boundary calculations from portable reduced records; its `verify.py` checks the manifest and full-precision reference results.

F Offline decision-call microbenchmark

This microbenchmark reuses actual states from all 16 technically valid same-runtime comparison runs, including the B-HPA smoke with four unfinished offline jobs; the original technically invalid a1 is excluded with its failure retained. Independent recovery matches the full input and hidden pre/post-state of all 29,580 policy ticks, including guard

latched IDs and B-HPA's complete queue history. Remaining work is the frozen policy's surrogate, not GPU progress inferred from future completions.

Before timing, a fixed rule chooses the earliest available tick per run and queue stratum: empty, intermediate, or maximum observed queue size; B-HPA also separates update and hold ticks. Empty strata stay empty. The resulting 60 snapshots are retrospective cases, not random production or held-out samples. Each receives 100 warmup and 1,000 timed calls of the original sealed `adapter.decide`, including its existing validation. Every call uses a newly restored adapter and is checked against the original decision; restoration, copying, and checks are outside `perf_counter_ns` timing. All 60,000 timings, including slow values, are retained; they are repeated measurements of 60 states, not 60,000 independent physical experiments.

The host is an Intel Xeon E5-2682 v4 (2.50 GHz), eight logical CPUs, with CPython 3.13.5, distinct from the GPU execution environment. CPU use is not exclusive; affinity is unchanged and frequency policy is unavailable. Table 20 reports all groups. Formal guard's largest single-snapshot p99 is 120.105 μs in steady and 92.39661 μs in recovery; its largest individual observed times are 157.491 and 841.877 μs. The latter is retained rather than hidden by a quantile. These are neither pooled p99s nor production-run tail estimates.

The interval excludes `dispatch_kind`, admission, the full controller loop, replay/restoration, logging, I/O, worker and GPU execution. It measures offline decision-call cost on this CPU, not complete-controller or online end-to-end overhead, and cannot establish absence of online tail interference. The recovery and timing sources are `ANALYSIS/E2_RECOVERY_v1/` and `ANALYSIS/E2_BENCHMARK_v1/` under `docs/maxopt_v4_r3_p5_20260929/`; the latter preserves all raw samples. `REVIEW_E2.json` binds the independent audit, and `P6_E2_TABLE_MANIFEST.json` binds this presentation.

G POLICY DEPLOYMENT EVIDENCE

The deployment study handoff is independently verified by `tools/verify_portable_32_cpu.py`. Its package SHA256 is `079e2b7e146b1706c2500ac9a3c4a5ff0c4310885cc013907e9b86c754399aa3`; the extracted package contains 32 cells, 36 attempts, 20 strict P1+ results, 12 scientific negative results, and four retained technical-invalid attempts. No checkpoint was retrained or selected after physical execution. The all2 reference is one cell per window; each PPO cell is a repeated execution of the same frozen checkpoint, not a new training seed.

The physical package stores both Lookup-69 and Lookup-30 predictions for each cell. Comparing the frozen prediction labels with the final physical labels gives 32/32 feasibility-direction agreement for Lookup-69 and 20/32 for Lookup-30. Window-cost MAE is 0.0804 and 0.4657 scenario USD, respectively; these metrics use the same registered observation-window ledger and do not estimate provider billing. The Lookup-30 prediction marks PPO-5 feasible in both windows, while all six physical repeats fail the offline deadline. This is a false-positive deployment case, not a reason to remove the negative cells.

The package also retains the exact attempt history, source and tensor hashes, request terminals, controller events, lifecycle manifests, budget receipts, and CPU recomputation for every cell. The four technical-invalid attempts remain in the audit trail and are excluded from scientific repeat counts. The evidence is bounded to two locked windows, two A100 GPUs, one Qwen model, five checkpoints, and the declared synthetic-API scenario ledger.

G.1 Frozen PPO training and selection contract

The capacity policy uses a 30-dimensional observation, a shared MLP with two 256-unit ReLU layers, three capacity logits, and a scalar value head. The four-logit offline head is masked to its sole legal action. Training runs in CPU replay with guard disabled, using Lookup-69 or Lookup-30 and 2,100-step episodes and rollouts. Adam uses learning rate 3×10^{-4} and numerical epsilon 10^{-5}; PPO uses discount 0.9995, GAE parameter 0.999, clip 0.2, four update epochs, minibatches of 64, value coefficient 0.5, entropy coefficient 0.01, and gradient norm limit 0.5. Advantages are normalized and value loss is clipped; the learning rate is constant. These settings are recovered from the frozen configuration, not retuned for the deployment results.

Training uses three development windows with prefix `train_` and suffix `_v2` for the burst-offline, recovery, and steady regimes. Selection uses the corresponding three `validation_*_v2` windows with deterministic Lookup-30 replay, guard disabled, and P1+ required separately in every window. Eligible checkpoints are ordered by equal-window mean cost, checkpoint index, and seed. The deployment study reuses the two previously locked mixed-policy recovery and steady inputs; these are distinct files from the selection inputs. The training contract specifies paired seeds 101–105; Table 22 gives the actual selected checkpoints. The accompanying `PPO_TRAINING_CONTRACT.json` records the configuration, selection-input hashes, and checkpoint identities. Table 22 maps the paper labels to the archived identifiers. In those identifiers, `r2` denotes the second eligible checkpoint, `s101` the seed-101/episode-40 matched pair, and `s103` the best eligible checkpoint within seed 103; the reference is deployed as `ppo_C30_raw`.

Table 20. Offline `adapter.decide` timings, in microseconds. Median range spans the per-snapshot medians; max p99 is the largest per-snapshot p99, and max sample is the largest individual timing. N counts snapshots.

Kind	Window	Policy	N	Median range	Max p99	Max sample
smoke	steady	B-HPA	3	8.3695–13.0020	24.15627	36.124
smoke	steady	H+guard	3	39.7485–45.1900	61.39790	536.644
capacity	steady	all2	3	2.4390–2.4660	2.97401	19.672
capacity	recovery	all2	3	2.3900–2.4200	2.97409	19.565
comparison	steady	H+guard	9	43.2845–97.4360	120.10500	157.491
comparison	steady	B-HPA	15	12.1775–30.1865	45.79169	389.906
comparison	recovery	B-HPA	15	10.5785–25.8275	38.47390	49.562
comparison	recovery	H+guard	9	43.3315–75.1155	92.39661	841.877

Table 21. Policy deployment summary. Costs are means over three PPO repeats or the single all2 anchor in each window. A negative arm can have lower cost while still violating the strict deadline contract.

Arm	training environment	R P1+	S P1+	cost R	cost S	offline outcome
all2	anchor	1/1	1/1	4.200	4.200	120/120
PPO-2	Lookup-69	0/3	0/3	2.730	2.950	4–13 on time
PPO-1	Lookup-69	3/3	3/3	2.357	2.449	120/120
PPO-3	Lookup-30	3/3	3/3	2.073	2.084	120/120
PPO-4	Lookup-30	3/3	3/3	2.051	2.103	120/120
PPO-5	Lookup-30	0/3	0/3	1.899	2.090	106–114/120

Table 22. Paper labels and frozen checkpoint identities. Steps are those reached by the selected checkpoint. PPO-4 is the registered reference.

Label	Archive ID	Seed	Episode	Steps
PPO-1	`C-s101`	101	40	84,000
PPO-2	`C-r2`	103	100	210,000
PPO-3	`C30-r2`	101	25	52,500
PPO-4	`C30-ref`	101	40	84,000
PPO-5	`C30-s103`	103	75	157,500

G.2 Retrospective deployment lifecycle diagnosis

We diagnose all 32 accepted physical runs and their 24 frozen Lookup-69/Lookup-30 predictions without retraining or selecting new checkpoints. The two prediction configurations differ only in startup time (69.219 s versus 30 s). Window launch-to-ready time exceeds 60 s in 196 of 218 observed generations; the median of the 30 PPO runs' startup medians is 87.92 s. These are descriptive lifecycle measurements, not independent experimental replicates.

PPO-2 exposes the interaction with the 60 s cooldown, which starts at a target change rather than at readiness. In the first recovery run, its first window restart launches at 120.01 s, becomes ready at 223.89 s, and enters draining at 224.01 s without admitting work. The matched Lookup-69 prediction becomes ready at 189.22 s and drains at 190 s, also without admissions. Lookup-30 becomes ready at 150 s; although the policy proposes zero capacity, cooldown holds one GPU until 180 s, admitting eight offline requests. Across all six physical PPO-2 runs, 72 of 73 restarts longer than 60 s enter draining within two seconds of readiness. All 687 offline misses are already overdue before dispatch or remain undispatched at cutoff.

The other negative checkpoint has a milder queueing failure: 60 of PPO-5's 65 offline misses are overdue before dispatch, while all requests eventually complete. Long startup alone does not imply failure: PPO-1 passes despite 78 of its 84 window restarts exceeding 60 s, retaining capacity to clear its queues. PPO-3 and PPO-4 also pass, generally keeping a worker available after restarting. These archived trajectories explain a concrete failure pathway and why Lookup-69 detects its feasibility consequence. They do not establish a general ranking of startup models or attribute every physical timing difference to startup alone.

H Screening additional checkpoints

The screening study tests whether the replay model that agreed with the deployment study can screen additional checkpoints before deployment. From the existing training archive, we replay 90 previously undeployed candidates under Lookup-69 and Lookup-30 on both locked windows (360 CPU scenarios), without retraining. Lookup-69 accepts 67 candidates and rejects 23; Lookup-30 accepts 89. Acceptance requires P1+ on both windows. These are replay judgments, not measured success rates.

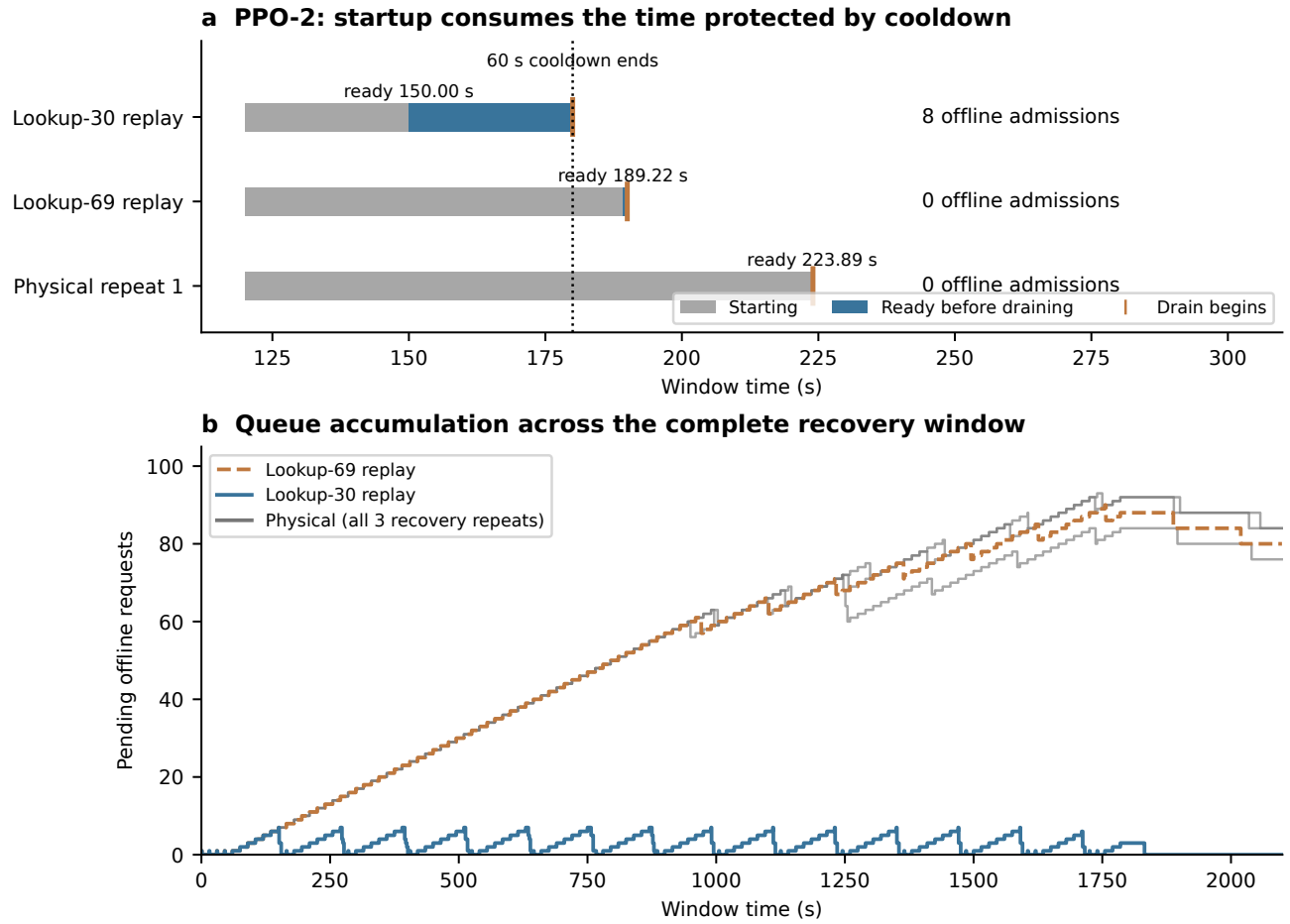


Figure 3. Retrospective PPO-2 recovery diagnosis. Top: the first window restart of the first formal negative run and its frozen Lookup-69/Lookup-30 predictions; admissions count offline requests dispatched during that instance generation. The dotted line marks cooldown expiry, measured from the target change. Bottom: pending offline queues in both predictions and all three physical recovery repeats. The text summarizes both windows and the passing-policy controls. No new physical runs are introduced.

Selection and execution. The selection rule uses Lookup-69 alone. Accepted candidates are ordered by mean window cost; rejected candidates by total offline and then online deadline misses, with checkpoint identity breaking ties. Within each group, identical complete target trajectories are represented once, and the first three representatives are selected. The resulting six Lookup-69 trajectories differ from the previously deployed policies. Table 23 maps the new labels to archived checkpoints; the selected set spans two training seeds. Lookup-30 accepts all six but does not determine their selection.

Each checkpoint executes twice per window, with one all2 reference per window: 24 policy runs and two references. A checkpoint is physically feasible only if all four executions satisfy P1+. Recovery and steady execute on separate matched dual-A100 hosts with the same model weights, vLLM 0.9.0 software stack, and accounting specifications. The recorded execution amendment changes work allocation, retaining the six checkpoints, inputs, predictions, and within-window run order. Window and host are therefore confounded; differences between the windows cannot be attributed solely to workload. The screening study remains separate from the deployment study's runtime group.

Table 23. Screening study checkpoint identities. Archived names encode training model, seed, and checkpoint index. Lookup-69 rejects PPO-9–PPO-11 because it predicts respectively 3, 6, and 7 offline misses in steady; all six pass recovery in replay.

Arm	Archived checkpoint	Lookup-69 decision
PPO-6	`C30_s105_e0065`	accept
PPO-7	`C30_s105_e0035`	accept
PPO-8	`C30_s102_e0005`	accept
PPO-9	`C30_s102_e0080`	reject
PPO-10	`C_s105_e0080`	reject
PPO-11	`C_s105_e0070`	reject

The launch records bind the frozen scientific files and CPU predictions; actions and replay outcomes match the CPU references. The external Git record was created after physical execution began, so this is a fixed-cohort confirmation, not an externally preregistered study. All 26 executions are technically valid, with no retries or replacement candidates.

Results and interpretation. All 26 runs satisfy P1+. Thus Lookup-69 correctly accepts three checkpoints but falsely rejects the other three; Lookup-30 accepts all six correctly on this cohort. There are no physically infeasible checkpoints, and Lookup-30 has no rejected candidates, so neither failure-detection performance nor Lookup-30's rejection accuracy can be estimated. The selected cohort and repeated runs also do not provide a population screening-accuracy estimate.

Across the 24 policy runs, feasibility agreement is 18/24 for Lookup-69 and 24/24 for Lookup-30: Lookup-69 predicts failure only in the steady cells of PPO-9–PPO-11. Cost MAE is 0.3112 for Lookup-69 and 0.0163 for Lookup-30 (Table 24); the two all2 references are excluded from this comparison. These are scenario costs, including the declared synthetic-API route, rather than measured provider charges. The outcome reverses the model ordering observed in the deployment study on a different cohort and execution context; it does not establish that either fixed model is generally preferable.

In this execution context, the 76 completed in-window launch-to-ready intervals have median 34.78 s (range 32.52–93.01 s), closer to Lookup-30's 30 s assumption than Lookup-69's historical 69.219 s value. Two additional late launches are terminated at the observation cutoff before readiness and are excluded from this duration summary as right-censored observations; their window occupancy is charged. These observations do not isolate startup time as the cause of the gate errors. The screening study shows why a screening decision itself requires physical validation: a model successful on an earlier cohort can reject feasible policies in a later deployment context.

I TRANSFER TO SGLANG

The engine study evaluates the shared policy and accounting contract on a second serving stack. It reuses H, H+guard, two deployment checkpoints, and two screening checkpoints (Table 25), with no retraining. Among the screening study's historically accepted and rejected candidates under Lookup-69, the lowest mean physical-cost candidate in each group supplies the two representatives. This fixed, nonrandom cohort tests transfer of existing policies and predictions.

Backend and execution. The deployment uses Qwen2.5-7B-Instruct on two A100 80GB PCIe GPUs, one independent TP1/PP1 SGLang 0.4.8 instance per GPU, FP16, model length 4096, static-memory fraction 0.8, chunked-prefill size and maximum prefill tokens 16,384, maximum running requests 1,024, and four local slots. CUDA graphs and radix caching are disabled. SGLang uses PyTorch 2.7.1 and Transformers 4.52.3; the CPU policy controller retains PyTorch 2.7.0. Native token-ID input avoids retokenizing the frozen workloads. A recorded compatibility fix supplies model EOS metadata when the tokenizer is not initialized, preserving minimum-output-token masking. The fix, input-token identities, output-budget checks, readiness probes, and verified process release are bound in the source and execution manifests.

Recovery and steady reuse deployment workloads: respectively 358 and 944 online arrivals, plus 120 offline arrivals per window. Each of six arms runs twice per window, with one all2 anchor per window: 24 policy runs and two anchors. The observation interval remains 2100 s, with the same deadline, routing, guard, and scenario-price definitions. Separate runtime identities are retained; the study is a transfer of the full serving stack, not an isolated engine-performance comparison.

Startup measurement and frozen predictions. Twenty calibration cycles launch the second instance while the first serves four concurrent 256-input/256-output-token background requests. Timing runs from launch request to parent-verified readiness. All 20 cycles complete, with median 27.5675401645 s and range 26.713–29.165 s; setup timing is recorded separately. Lookup-SG substitutes that unrounded median for Lookup-69's 69.219185665 s startup proxy. Lookup-30 uses 30 s. All three share the inherited service lookup, 7.059 s shutdown proxy, policy identities, and accounting rules; the guard's safety estimates remain unchanged. This calibrates startup, not the full engine service model.

Design and compatibility checks precede calibration; the GPU-side design check is local. The 42 predictions (three models, seven policies including all2, two windows) and measured startup parameter are frozen and independently accepted on CPU before the formal runs. Physical repeats reuse the same prediction, so the 24 comparisons per model represent 12 unique policy/window predictions rather than 24 independent samples.

Results. All 26 final runs are technically valid. Both all2 anchors meet every deadline, occupy 4200 GPU seconds, and cost 4.2000 scenario USD each. All online requests are timely. H leaves one offline job unfinished in each steady repeat; all other policy runs complete all work on time. Tables 26 and 27 retain both repeats of every policy/window pair. P1+ failures remain in the agreement and cost-error denominators. Mean absolute error uses total scenario cost,

Table 24. Screening study window costs in scenario USD, with both physical repeats shown. R and S denote recovery and steady. Predictions include any modeled deadline penalty; no physical run incurs one. Each all2 reference satisfies P1+ with cost 4.2000. Costs cover the 2100 s observation window; setup and subsequent cleanup are recorded separately.

Arm	Window	Lookup-69 prediction	Lookup-30 prediction	Physical 1	Physical 2	P1+
PPO-6	R	1.9351	1.9351	1.9269	1.9270	2/2
PPO-6	S	1.9767	1.9668	1.9575	1.9579	2/2
PPO-7	R	1.9102	1.8709	1.8698	1.8445	2/2
PPO-7	S	2.0171	1.9517	1.9236	1.9275	2/2
PPO-8	R	1.9720	1.9468	1.9315	1.9322	2/2
PPO-8	S	2.0323	1.9760	1.9617	1.9618	2/2
PPO-9	R	1.9650	1.9621	1.9523	1.9524	2/2
PPO-9	S	2.0689	2.0180	2.0171	2.0172	2/2
PPO-10	R	2.6433	3.6908	3.6664	3.6667	2/2
PPO-10	S	3.0121	3.6548	3.6242	3.6243	2/2
PPO-11	R	2.8257	3.7802	3.7660	3.7580	2/2
PPO-11	S	3.0603	3.9023	3.8758	3.8769	2/2

Table 25. Frozen PPO identities reused in the engine study. PPO-1/PPO-2 come from the deployment study and PPO-7/PPO-9 from checkpoint screening. Rule parameters and guard safety estimates are unchanged.

Label	Training environment	Seed / checkpoint
PPO-1	Lookup-69	101 / 40
PPO-2	Lookup-69	103 / 100
PPO-7	Lookup-30	105 / 35
PPO-9	Lookup-30	102 / 80

including the offline-miss penalty, consistently with the main paper.

Lookup-69 falsely rejects eight feasible runs: both steady H+guard runs, all four PPO-2 runs, and both steady PPO-9 runs. At arm level, requiring P1+ in all four executions, Lookup-69 accepts two feasible arms and rejects four, three of which are feasible. Lookup-30 and Lookup-SG each accept the five feasible arms and reject H. Thus local startup calibration and Lookup-30 give identical feasibility judgments here. Their total-cost MAEs are 0.0427 and 0.0468, respectively, versus 0.5420 for Lookup-69. Excluding deadline penalties from both prediction and observation gives operating-cost MAEs of 0.0427, 0.0468, and 0.3411 in the same order. The cost-error difference is therefore not solely a penalty effect.

H+guard passes all four runs and averages 920.795 occupied GPU seconds, 78.08% below the all2 anchors; individual-run reductions span 75.21–80.90%. These are window resource savings under the synthetic-API contract. H's low occupancy alone would hide its two deadline failures.

Across valid formal runs, 255 of 256 ordinary in-window launches reach readiness. Their median is 28.052 s, p95 43.094 s, and range 26.112–94.872 s. The remaining launch is stopped by H+guard after 24.002 s before readiness; its occupancy is charged, and it is excluded from completed-startup statistics. Initial setup and invalid attempts are excluded from this distribution. The calibration captures the typical startup scale, while the physical record preserves its variability.

Retrospective startup sensitivity. We vary only Lookup-SG's startup proxy over 18 values from 20 to 69.219 s, keeping policies and all other replay parameters fixed. Against the same 24 physical outcomes, all nine sweep values from 20 through 41 s retain 24/24 agreement; the 42 s test gives 22/24 (Table 28). At Lookup-69's original startup value, replay reproduces its eight false rejections. Exact agreement is therefore not specific to the measured median or the 30 s setting. The source bundle includes the full grid and per-run comparisons.

Technical attempts and verification. There are 28 attempts for 26 cells. PPO-1's two recovery executions each encounter a control-loop timeout on their first attempt and are repeated once with unchanged configurations under recorded authorization. The historical causes remain unresolved; the failed attempts are retained. An additional environment-retry provision was recorded during execution but never used. H's two deadline-violating runs are not retried. The CPU receiver verifies all file manifests and recomputes request outcomes and resource/cost ledgers using the original frozen analyzer; all 26 ledgers agree with the GPU return. The supplement's `E4_RESULT_SPECIFICATION.json` records the source bindings, parameters, and result definitions.

Table 26. Engine study predicted and observed total window costs in scenario USD. R/S denote recovery/steady. A dagger marks predicted or observed P1+ failure; unmarked entries satisfy P1+. Both all2 anchors cost 4.2000 and satisfy P1+.

Arm	Window	Lookup-69	Lookup-30	Lookup-SG	Physical 1	Physical 2
H	R	1.8007	1.0215	1.0186	0.9799	1.1618
H	S	2.1624†	1.3655†	1.3539†	1.3107†	1.2926†
H+guard	R	1.8007	1.0215	1.0186	1.0266	0.9678
H+guard	S	2.3375†	1.4256	1.4139	1.3930	1.3470
PPO-1	R	2.1451	2.1696	2.1674	2.1119	2.1371
PPO-1	S	2.2972	2.2966	2.2908	2.2300	2.2390
PPO-2	R	2.6994†	1.4054	1.4025	1.3410	1.3429
PPO-2	S	3.0270†	1.6321	1.6211	1.5520	1.5455
PPO-7	R	1.9102	1.8709	1.8704	1.8397	1.8390
PPO-7	S	2.0171	1.9517	1.9487	1.9248	1.9222
PPO-9	R	1.9650	1.9621	1.9621	1.9558	1.9551
PPO-9	S	2.0689†	2.0180	2.0180	2.0202	2.0191

Table 27. Engine study physical resource use and offline on-time counts, shown for both repeats. All online arrivals are on time; each offline denominator is 120. H's two misses are unfinished jobs at cutoff. Occupancy covers the registered window, including startup, draining, and stopping until verified release.

Arm	Window	GPU seconds 1	GPU seconds 2	Offline 1	Offline 2
H	R	814.275	990.208	120/120	120/120
H	S	951.587	937.009	119/120	119/120
H+guard	R	836.218	802.155	120/120	120/120
H+guard	S	1041.078	1003.729	120/120	120/120
PPO-1	R	2004.565	2026.170	120/120	120/120
PPO-1	S	2015.445	2016.216	120/120	120/120
PPO-2	R	1149.451	1150.762	120/120	120/120
PPO-2	S	1129.373	1133.962	120/120	120/120
PPO-7	R	1757.456	1759.098	120/120	120/120
PPO-7	S	1765.317	1755.676	120/120	120/120
PPO-9	R	1951.793	1951.098	120/120	120/120
PPO-9	S	1982.853	1982.961	120/120	120/120

J VALIDATION ACROSS ARRIVALS AND MODEL DEPLOYMENTS

The arrival study holds engine calibration fixed while changing arrivals; the 14B study remeasures lifecycle timing for a 14B deployment. Both use the same 2100 s window, deadlines, four local slots per GPU, and scenario ledger. The original predictions precede physical execution, and repeats share a prediction; the additional 30 s contrast below is retrospective. Table 29 reports policy cells, excluding full-capacity anchors.

J.1 Fixed calibration on additional arrival windows

The arrival study retains Qwen2.5-7B-Instruct/SGLang 0.4.8 and the engine study's backend settings, including native token input. It reuses H, H+guard, PPO-1, and PPO-2 without policy search or PPO retraining. Each arm runs twice on three constructed windows, plus one all2 anchor per window: 27 cells, 24 of which are policy cells. Lookup-69, Lookup-30, and Lookup-SG retain startup values of 69.219, 30, and 27.568 s and the inherited service lookup. There are 12 unique policy/window predictions per model.

The windows are steady, middle-burst, and late-burst, with 1074, 389, and 351 online arrivals, respectively, plus 120 offline arrivals each. They use the existing BurstGPT-derived source and fixed thinning pipeline. Their source rows do not overlap the canonical physical-evaluation windows or the training/validation windows of these PPO checkpoints. Earlier full-stream CPU replay did include these source times, and the prompt corpus is reused: this tests additional constructed arrivals, not an independent data source or prompt holdout.

All 27 cells pass P1+. Lookup-69 falsely rejects ten policy runs (five distinct policy/window predictions), Lookup-30 rejects two (the two steady H repeats), and Lookup-SG rejects none. False-accept rates are undefined because there are no physically infeasible cells. The measured-

Table 28. Selected values from the retrospective CPU sweep, plus the measured-median reference (marked ∗). Startup values are rounded for display. Each row reuses the same 24 engine policy runs; the sweep tests discrete values, not a continuous tolerance interval.

Startup (s)	Agreement	False accepts	False rejects
20	24/24	0/2	0/22
27.568*	24/24	0/2	0/22
30	24/24	0/2	0/22
41	24/24	0/2	0/22
42	22/24	0/2	2/22
69.219	16/24	0/2	8/22

startup model has slightly higher total-occupation MAE than Lookup-30 (96.92 versus 94.34 GPU seconds) but lower cost and trajectory errors overall. The trajectory improvement is concentrated in steady; the two burst windows have slightly higher trajectory errors than Lookup-30. Per-window and per-cell values accompany the source bundle.

No guard run in the arrival study has a risk-active tick, target raise, or reservation; minimum queued-offline slack is 152–153 s. These runs test fixed-parameter prediction transfer, not an additional guard rescue.

J.2 Startup contrast in a 14B deployment

The 14B study uses Qwen2.5-14B-Instruct with vLLM 0.9.0, PyTorch 2.7.0+cu126, and Transformers 4.51.1 on two A100 80GB GPUs. Each GPU hosts an independent TP1 instance, using the main paper's FP16, eager execution, memory, model-length, batching, and four-slot settings. The recovery and steady input files are byte-identical to the engine study's locked inputs. H and H+guard each run twice per window, alongside two all2 anchors per window: 12 cells, with eight policy cells and four unique policy/window predictions per model.

Calibration uses synthetic development requests before formal prediction or execution. The service grid crosses input lengths 128/512/2048, output lengths 32/128/256, and concurrency 1/4, plus the 256/256, concurrency-four offline shape. Each shape has three repeats per GPU; the lookup takes each GPU's median batch-maximum parent-clock duration, then the median across the two GPUs. The offline shape measures 5.844 s. Twenty scale-out cycles (ten per GPU) launch one instance while the other has four admitted 256/256-token background requests. Launch-to-ready median is 46.067 s, range 45.814–52.352 s. Setup is separately 48.486 s (maximum of two initial launches); median shutdown is 1.980 s.

14B-Hist reuses the historical 69.219 s startup proxy; 14B-Local uses the new 46.067 s median. Their service, setup, shutdown, policy, and guard settings are identical. The guard uses the measured service lookup and a separately frozen 47.765 s startup safety estimate (linear p95 of the 20 cycles), with the 52.352 s maximum for overdue-start handling and the existing 5 s margin. These sampling estimates carry no coverage guarantee. The source bundle records the unrounded parameters and complete service lookup.

All 12 cells are technically valid and all four anchors pass P1+. H leaves seven offline requests unfinished in each recovery repeat and one in each steady repeat; all four H+guard runs satisfy P1+. 14B-Hist falsely accepts all four failed H runs. 14B-Local correctly classifies both arms in every repeat, while lowering occupied-resource MAE from 423.35 to 68.46 and trajectory L1 from 1049.53 to 338.15 GPU seconds. This single-field predictor contrast demonstrates the value of local startup measurement given the shared 14B service calibration. It does not compare reusing versus remeasuring the service curve, or isolate a causal effect of model size, engine, or hardware.

Two guarded runs (one per window) contain 47 risk-active/reservation ticks and one target raise each; the other two have no guard activity. The active runs have minimum queued-offline slack of 13 s and four/one reserved offline dispatches in recovery/steady; inactive runs have minimum slack of 126/129 s. Thus the observed state trajectory, rather than the window label alone, determines activation.

A fixed 30-second contrast. To compare local measurement with the existing simple startup assumption, we retrospectively replay 14B-30: the same 14B service, setup, shutdown, and guard parameters, with startup set to 30 s. This adds four policy/window predictions reused across the eight existing physical runs, plus two separate all2 predictions; it adds no GPU run or parameter search. 14B-Hist and 14B-Local reproduce their original frozen predictions. 14B-30 agrees on 6/8 policy runs, with two false accepts (both steady H repeats) and no false rejects. Its occupied MAE, cost MAE, and trajectory L1 are 269.12 GPU seconds, 0.2609 scenario USD, and 578.92 GPU seconds, between 14B-Hist and 14B-Local in Table 29. All2 is unchanged across the three models. This diagnoses the existing deployments; 14B-30 was not frozen before those physical runs.

From startup to tail deadlines. All three H replays first restart at 195 s in recovery and 138 s in steady. At cooldown expiry (255/198 s), 14B-Hist is still starting: its queue-plus-running pressure is 13/14, so H raises the target to two GPUs. 14B-Local is already serving; pressure is 5/8 and the target remains one. 14B-30 has pressure one in both windows. These decisions alter later expansion and shutdown timing.

In recovery, physical H releases its last GPU near 1692 s, leaving seven later offline arrivals (1695–1785 s) below H's scale-out threshold of nine. They remain undispatched through the 2100 s cutoff. In steady, the final 1785 s offline arrival is similarly stranded after scale-down near 1780 s. 14B-Local predicts the same failed request IDs in both repeats of each window. 14B-Hist's later shutdown lets it finish those tail requests; 14B-30 leaves six in recovery but finishes the last steady request, explaining its false accepts.

14B-Local's guarded replay restarts capacity for the stranded queues at 1936 s in recovery and 2026 s in steady. The two risk-active physical guard runs trigger at those same ticks and complete the tail work. The other two guarded runs finish without risk activation because their earlier trajectories differ. Some physical-state mismatches also precede the first restart, involving service or release timing. Thus the fixed-field replay contrast identifies a startup feedback path; it does not attribute every physical discrepancy to startup, or every guarded success to a guard raise. Exact event and request chains accompany the reduced diagnostic in `artifact/e6_diagnostic/`.

Verification and reduced evidence. Both campaigns finish without skipped cells or technical retries. All four 14B deadline failures remain included. CPU reanalysis reproduces the sealed accounting and analysis files. The 14B study required a recorded identity-parser adaptation for distribution metadata `2.7.0` versus runtime `2.7.0+cu126`; CUDA and source identities remained checked. A prepared external clock-recovery extension was not triggered in either campaign. The source bundle's `artifact/e5_e6/` contains per-cell comparisons, per-window summaries, guard diagnostics, and 14B calibration parameters; `python verify.py` recomputes Table 29 from these reduced comparisons. This verifies summary aggregation, not raw GPU execution. The full machine archives remain separately retained. The arrival study/14B request and lifecycle projections are included in the expanded paired-trace corpus; the retrospective 14B-30 diagnostic is separate from its frozen predictions.

K QUANTIZED DEPLOYMENT VALIDATION

The AWQ study extends the closed-loop validation contract to one quantized serving configuration. It is a deployment-context validation, rather than a quantization-only causal comparison: the quantized weights, kernel, loading path, and runtime are all part of the new configuration. The policies, workloads, deadlines, accounting prices, and guard rule remain those fixed in the main study.

Fixed configuration. We use Qwen2.5-7B-Instruct-AWQ with vLLM 0.9.0 and the AWQ-Marlin backend on one host with two A100 80GB PCIe GPUs. Each GPU runs an independent TP=1 instance with FP16 activations, four local slots, a 4096-token context, eager execution, chunked prefill, and no prefix cache. The preceding precheck measured 19 service shapes (three repeats per shape on each GPU), 20 loaded scale-out cycles, and separately measured setup and release. It froze three CPU predictors that share the new service and guard parameters and differ only in startup: AWQ-Hist reuses the historical 69.219 s proxy, AWQ-30 uses 30 s, and AWQ-Local uses the measured 25.810 s median. Only AWQ-Local drives the physical controller; the other two are counterfactual CPU replays.

Physical matrix. The formal matrix has two locked windows, recovery and steady, and three policies: all2, H (the hysteresis policy), and H+guard. Each cell is repeated twice for 12 physical runs. The four all2 cells are capacity anchors. The remaining eight cells are compared with the frozen AWQ-Local prediction made before execution. A run is technically valid only after worker identity, request terminals, lifecycle cleanup, and the resource ledger pass independent CPU checks. No technical retry was used.

Results and interpretation. All four all2 anchors pass P1+, and all eight policy cells are technically valid. AWQ-Local predicts all eight policy outcomes correctly: the six feasible cells pass and the two H/steady cells fail in both repeats. Those two physical failures are scientific negative results, not technical failures: every online request is on time, while 119 of 120 offline requests meet their deadlines and one remains unfinished at the 2100 s cutoff. H+guard completes the same steady window in both repeats. The frozen policy target matches the physical target at all 2,100 ticks in every policy cell. Across the eight policy cells, mean absolute error is 2.745 GPU seconds for occupied resource and 0.00436 scenario USD for total cost; the corresponding mean trajectory L1 error is 3.143 GPU seconds.

Table 31 compares all three frozen predictors against the same physical runs. AWQ-Hist falsely rejects both feasible H+guard steady repeats; AWQ-30 matches all eight outcomes with 3.09 times AWQ-Local's occupancy MAE. Thus historical startup reuse can change feasibility judgments even with shared local service estimates. All three correctly predict the H/steady failures.

The AWQ archive retains the calibration records, frozen predictions, raw controller and worker events, per-cell request terminals, lifecycle ledgers, and independent CPU audit files. The reduced predictions and ledgers are recomputed by `artifact/awq_formal/verify.py`. These files are separate from the public 197-run corpus counts in Appendix A; the AWQ study should be counted as an additional deployment-context extension when the archive is made public.

Table 29. Arrival study/14B policy-cell results. FA/FR denote false accepts/rejects, divided by physically infeasible/feasible cells; a dash denotes an empty denominator. Resource errors are GPU seconds and total-cost MAE is scenario USD, including deadline penalties. Means weight windows equally. Repeats share predictions.

Study	Replay	Agreement	FA	FR	Cost MAE	Occupied MAE	Trajectory L1
Arrivals	Lookup-69	14/24	—	10/24	0.7082	500.90	1268.65
Arrivals	Lookup-30	22/24	—	2/24	0.0930	94.34	307.16
Arrivals	Lookup-SG	24/24	—	0/24	0.0897	96.92	250.39
14B	14B-Hist	4/8	4/4	0/4	0.4105	423.35	1049.53
14B	14B-Local	8/8	0/4	0/4	0.0668	68.46	338.15

Table 30. AWQ-Marlin physical results, averaged over two repeats. The prediction and physical columns report P1+ passes over the two repeats; offline counts use 120 arrivals per repeat. Occupancy (GPU seconds) and cost (scenario USD) are physical means.

Window	Policy	Pred.	Phys.	Offline	GPU sec.	Cost
Recovery	H	2/2	2/2	120/120	788.505	0.9536
Recovery	H+guard	2/2	2/2	120/120	788.623	0.9537
Steady	H	0/2	0/2	119/120	925.882	1.2698
Steady	H+guard	2/2	2/2	120/120	977.267	1.3137

Table 31. Frozen AWQ predictions versus eight policy runs, excluding four capacity anchors. FA/FR count false accepts/rejects. Occupancy MAE is in GPU seconds; cost MAE is in scenario USD. Four policy/window predictions per model are each reused for two physical repeats.

Predictor	Correct	FA	FR	Occ. MAE	Cost MAE
AWQ-Hist	6/8	0	2	868.054	0.83361
AWQ-30	8/8	0	0	8.495	0.02247
AWQ-Local	8/8	0	0	2.745	0.00436

L CONSEQUENCES FOR DEPLOYMENT CHOICES

The preceding comparisons measure classification and cost-prediction errors. Here we retrospectively apply the same deployment-choice rule to each campaign's recorded non-anchor candidate pool: require predicted P1+ in *every* window, then select the policy with the lowest mean predicted window cost. Windows receive equal weight. Archived policy ID breaks exact cost ties at the released data precision; all tied minimizers remain in the artifact. Physical results do not enter this selection rule.

For evaluation, a selected policy is feasible only if every recorded physical repeat passes P1+. Its observed cost first averages repeats within each window, then averages windows equally. Let c_* be the lowest such observed mean among candidates feasible in all their runs. We report the cost gap $100(c_{\text{selected}}/c_* - 1)$ only for feasible selections. This is a comparison within the tested pool, not a global optimum; all2 capacity anchors are excluded. Each campaign retains its own workload, runtime and price contract.

Table 32 makes the practical consequences explicit. The deployment study's Lookup-30 has a unique cost-minimizing candidate that fails every physical repeat. In the engine study and the arrival study, Lookup-69 excludes H+guard, the lowest-cost feasible policy in those measured pools; its chosen policies cost 58.95% and 94.13% more, respectively. Lookup-30 selects H+guard in both. These are opportunity costs under the declared synthetic-API ledger, not commercial savings attributed to a new optimizer. The arrival study's Lookup-SG ties H and H+guard in prediction; their 0.31% observed cost difference is descriptive and does not establish a reliable ranking. The 14B study's 14B-Hist also ties these policies, so a failing choice is possible under the stated rule rather than inevitable under every tie-break.

The candidate pools contain five, six, six, four and two policies in deployment, screening, engine, arrival, and 14B studies, respectively. The screening study therefore compares its six physically evaluated checkpoints, not all 90 screened candidates. This reanalysis adds no physical run or prospective selection test and does not choose a universally preferred replay model. The source bundle's `artifact/decision_utility/` reconstructs the choices and table from reduced per-record metrics; the paired corpus provides the underlying request and accounting records.

L.1 Price robustness of deployment choices

We reprice the same recorded events at GPU and synthetic-API price multipliers in $\{0.5, 1, 2\} \times \{0.5, 1, 2\}$, keeping startup, shutdown, and deadline-miss charges unchanged. At each point, we repeat the prediction-only selection and recompute the lowest-cost physically feasible comparator within the same pool. Table 33 summarizes all 108 choices; ranges are grid extrema, not confidence intervals or new executions.

The deployment study's Lookup-30 uniquely selects the failing PPO-2 throughout the grid. Lookup-69 selections in the engine and arrival studies remain feasible but

Table 32. Retrospective policy choices from the original frozen predictions. Costs are scenario USD; a dash excludes an infeasible selection from the feasible-cost comparison. P1+ counts repeated physical executions, not independent choices. † marks a prediction-cost tie between H and H+guard, resolved by archived ID to H. The arrival study's two tied policies are physically feasible; the 14B study's H fails whereas H+guard passes.

Study	Replay	Selected policy	Physical P1+	Observed cost	Feasible cost gap (%)
Deployment	Lookup-69	PPO-4	6/6	2.0769	0.00
Deployment	Lookup-30	PPO-2	0/6	2.8398	–
Screening	Lookup-69	PPO-6	4/4	1.9423	2.70
Screening	Lookup-30	PPO-7	4/4	1.8913	0.00
Engine	Lookup-69	PPO-7	4/4	1.8814	58.95
Engine	Lookup-30	H+guard	4/4	1.1836	0.00
Engine	Lookup-SG	H+guard	4/4	1.1836	0.00
Arrivals	Lookup-69	PPO-1	6/6	2.1655	94.13
Arrivals	Lookup-30	H+guard	6/6	1.1155	0.00
Arrivals	Lookup-SG	H†	6/6	1.1189	0.31
14B	14B-Local	H+guard	4/4	1.6019	0.00
14B	14B-Hist	H†	0/4	1.5149	–

Table 33. Deployment-choice sensitivity over all nine price combinations. Feasible counts refer to price combinations whose selected policy passes all recorded physical repeats. † retains the H/H+guard prediction tie: ID order selects H, but H+guard is also a minimizer.

Study	Replay	Selected policies	Feasible price cells	Cost gap range (%)
Deployment	Lookup-69	PPO-4, PPO-1	9/9	0.00–11.04
Deployment	Lookup-30	PPO-2	0/9	–
Screening	Lookup-69	PPO-7, PPO-6	9/9	0.00–3.11
Screening	Lookup-30	PPO-7, PPO-6	9/9	0.00
Engine	Lookup-69	PPO-7	9/9	16.56–80.17
Engine	Lookup-30	H+guard	9/9	0.00
Engine	Lookup-SG	H+guard	9/9	0.00
Arrivals	Lookup-69	PPO-1	9/9	41.93–119.65
Arrivals	Lookup-30	H+guard	9/9	0.00
Arrivals	Lookup-SG	H†	9/9	0.27–0.33
14B	14B-Local	H+guard	9/9	0.00
14B	14B-Hist	H†	0/9	–

costlier than the tested feasible minimum. The deployment study's Lookup-69 and both screening predictors change their choices with prices, confirming that the analysis reselects policies rather than simply repricing a fixed choice. The arrival study and 14B ties persist throughout: the arrival study's small H/guard cost difference does not establish a reliable ranking, and the 14B study's failing choice depends on the tie-break. The source bundle's `artifact/decision_price/` retains every price point, source-bound cost component, and selection, and exactly recovers the nominal-price results above. Fixed-event repricing does not model policies retrained or redeployed under new prices.

M Price sensitivity details

We reprice H+guard and all1 over 27 window–GPU–API price combinations, keeping their event streams fixed. At the reference prices the three window savings are 21.96%, 48.19%, and 58.08%; the sign can reverse when GPU capacity is cheap and API use is expensive. The only negative cell is steady at GPU multiplier 0.5 and API multiplier 2.0, where savings are -39.59%. The steady break-even API multiplier at half-price GPUs is 1.004. Recovery and burst remain positive across the tested grid, but this is a finite sensitivity analysis, not a provider billing forecast.

N Retrospective lifecycle-trajectory diagnosis

This analysis reuses all nine accepted fixed-H physical runs and the twelve unique Share/Lookup-69/Lookup-30/Local predictions from the frozen held-out evaluation: three windows, three physical repeats per window, and four model comparisons per run. The resulting 36 pairings are not new experiments or independent prediction samples. We neither

Table 34. Price-grid savings (percent) for API multipliers 0.5/1.0/2.0. Rows are GPU multipliers 0.5/1.0/2.0; the final column is the break-even API multiplier.

Window	GPU	API .5	API 1	API 2	break-even
steady	0.5	20.05	0.17	-39.59	1.004
steady	1.0	31.90	21.96	2.07	2.104
steady	2.0	37.84	32.86	22.92	4.304
recovery	0.5	46.51	41.13	30.37	4.822
recovery	1.0	50.88	48.19	42.81	9.953
recovery	2.0	53.07	51.73	49.04	20.216
burst-offline	0.5	56.53	53.75	48.18	10.647
burst-offline	1.0	59.47	58.08	55.29	21.841
burst-offline	2.0	60.95	60.25	58.86	44.227

refit a model nor replace the original M1 endpoint. The artifact binds the consumed records to their original acceptance hashes and rechecks all 44,100 decisions across the 21 unique traces.

N.1 Definitions and event alignment

Time is relative to each window start, on $[0, 2100)$ seconds. Physical occupied capacity counts each generation from launch until verified release, clipped to that interval; simulated capacity includes every non-off state, including stopping. Let $n_m(t)$ and $n_r(t)$ be predicted and observed occupied GPU counts. For model m and repeat r, we distinguish

$$A_{m,r} = \left| \int_0^{2100} (n_m(t) - n_r(t))\, dt \right|, \tag{2}$$

$$T_{m,r} = \int_0^{2100} |n_m(t) - n_r(t)|\, dt. \tag{3}$$

Both have units GPU $\cdot$ s. The first is the existing total-occupation absolute error; the second measures trajectory discrepancy without signed temporal cancellation. We integrate the piecewise-constant records exactly, without substituting one-second samples for continuous occupation. Table 36 reports equal-repeat means within each window; Table 37 preserves every pairing.

Targets are compared at all 2,100 one-second decision ticks. The first state-count difference compares the decision-time counts of active, starting, draining/stopping, and off devices; it is not the first difference of all hidden system state. Queue and running observations precede each decision. Routes are compared over all arrival IDs as local, synthetic API, or undispatched. For a request whose final routes differ, the recorded first dispatch evidence is a retrospective locator, not a time when both eventual routes were already known. Route differences need not follow target differences.

N.2 An endpoint discrepancy at the cooldown boundary

The frozen Lookup-69/Lookup-30 configurations differ only in the startup endpoint. Their first common starts and subsequent divergences are shown in Table 35. Lookup-30 becomes ready after 30 s, but targets stay equal until the 60 s cooldown expires. At that tick Lookup-69 remains starting; its queue-plus-running pressure exceeds the strict up-threshold of eight, and its target rises from one to two. Lookup-30 is ready with lower pressure and keeps target one. Both controllers have identical target/cooldown state before this decision; swapping the observed inputs swaps this one-step action. This diagnoses the frozen simulator contrast, not a new physical intervention or a replay of an alternative future trajectory. At the corresponding tick, each of the nine physical repeats has one active device, no starting device, and target one; that is an observational anchor. H does not read the predictor's startup parameter, and the guard's separately frozen safety-delay parameters are not intervened on here.

N.3 Counterexamples and temporal cancellation

The six burst/recovery physical repeats first differ from all four models in target at $t = 3$ s, before any window restart. At that tick the models observe no local running request while the physical controllers still do; completion observation has crossed a decision tick. These differences cannot be attributed to window-restart timing. Conversely, steady repeats 1 and 3 have exactly the same 2,100 targets as Lookup-30 despite 154 and 152 state-count difference ticks, 48 and 45 route differences, and nonzero occupied-resource errors. Readiness error therefore does not necessarily change a target.

The steady Local total MAE is 66.34 GPU $\cdot$ s, close to Lookup-30's 67.18, but its mean trajectory error is 1,390.50 versus 114.32 GPU $\cdot$ s. The same Local trace disagrees on 66.73% of target ticks on average. Accurate total occupation can thus hide substantial temporal mismatch; this diagnostic does not retroactively select a different model or alter M1. All Lookup-30 pairings retain nonzero occupation error. The largest Lookup-30 target residual is burst repeat 2 (637/2100 ticks); its longest mismatch spans $[1020, 1144)$ s. This example is selected by an explicitly post-hoc maximum residual rule after the full audit, alongside all no-target-change cases. The artifact exports every window's first common-start segment for all models and repeats, as well as the complete startup and unmatched-pair counts.

These observations support checking lifecycle states, actions, and resource trajectories alongside matched-service fit. Three controlled windows and nine repeats do not establish a universal startup/cooldown law, randomized physical

Table 35. First Lookup-69/Lookup-30 startup contrast in all three windows. Times are seconds from window start; pressure is queued plus locally running requests. The first target difference occurs at common start plus 60 s in each window.

Window	Common start	Lookup-30 ready	Lookup-69 ready	First target difference	Pressure Lookup-69/Lookup-30
Burst	195	225	264.219	255	13/1
Recovery	126	156	195.219	186	14/2
Steady	171	201	240.219	231	21/4

Table 36. All window/model means over three physical repeats. Total MAE is mean $A_{m,r}$; trajectory error is mean $T_{m,r}$, both in GPU · s. Target disagreement uses 2,100 ticks per pair; route disagreement uses all arrival IDs in that window, including undispatched requests.

Window	Model	Total MAE	Trajectory error	Target diff. (%)	Route diff. (%)
Burst	Share	234.48	457.24	15.17	27.95
Burst	Lookup-69	847.60	1369.86	49.63	27.61
Burst	Lookup-30	83.08	268.96	10.33	15.26
Burst	Local	70.55	237.76	10.33	15.71
Recovery	Share	244.08	421.31	11.71	27.75
Recovery	Lookup-69	864.20	1302.19	43.92	35.24
Recovery	Lookup-30	78.30	138.31	3.24	2.20
Recovery	Local	66.95	99.02	3.24	4.19
Steady	Share	1151.97	1517.83	59.40	42.84
Steady	Lookup-69	964.14	1621.50	73.68	38.87
Steady	Lookup-30	67.18	114.32	1.03	4.99
Steady	Local	66.34	1390.50	66.73	49.45

causality, or a general guard guarantee. Software-observed readiness is not a GPU-kernel timestamp, and the synthetic API retains its original quality and billing limitations.

O EXTERNAL-VALIDITY EVIDENCE

The external-validity study tests alternative target rules, a second model family, and a second arrival source under the same deadline and cost definitions. Each test is evaluated separately. The tables distinguish technical validity from strict qualification: the latter keeps the original request denominator, requires at least 99% online on-time completion, every offline job timely, and all requests completed without unfinished or censored work. A late offline completion is not an unfinished request, but neither qualifies. Table 10 accounts for all 93 original external-validity identities, including gates and six historical reuses. The same-runtime comparison adds 14 separately accounted formal identities (Section E); requests are not independent experimental repeats.

O.1 Alternative unguarded target rules

The alternative-controller test asks whether unguarded rules can also complete all work on time. B-HPA is an HPA-inspired queue-target controller with a stabilization interval, not an implementation of Kubernetes HPA. B-PRED applies a causal Holt queue predictor; B-MMC selects a capacity from the past arrival-rate estimate and the frozen Local service lookup. The three policies share the two-instance limit, lifecycle semantics, one-second observation tick, and 60-second action cooldown. None uses the held-out study's deadline guard, observes future arrivals, or trains a learned scheduler. No registered development candidate qualified, so all three use their predeclared center fallback; they are not presented as optimized best-in-family implementations. Section C.1 records their grids, locks, and semantic validation.

All three baselines complete all nine formal runs with timely online and offline work (Table 38). Their equal-window scenario costs are 2.2305, 4.0085, and 2.7861, respectively. Each exceeds the historical held-out study all1 reference, showing that timely completion alone does not identify the lowest-cost policy. All three succeed without the guard; adding the guard to these controllers remains untested. B-PRED's per-run online p99 is 1.58–2.70 seconds across these windows, whereas B-HPA and B-MMC reach 15–19 seconds in steady/recovery. Thus the higher-cost predictor also exposes a latency–resource trade-off; Table 11 reports each window's three-repeat range.

This comparison has a runtime boundary. The alternative-controller study's historical cells 001/002 and its r14 continuation belong to different runtime groups, and the new

Table 37. All 36 model/physical-repeat pairings. Errors are GPU · s; target differences are counts out of 2,100 ticks. A dash means targets never differ, not a missing record. Route counts retain the full arrival denominator. The twelve unique predictions are reused across the nine physical runs.

Window	Model	Rep.	Total error A	Trajectory T	Target diff.	First diff. (s)	Route diff./arrivals
Burst	Share	1	300.54	304.39	147	3	65/297
Burst	Share	2	103.59	764.07	662	3	117/297
Burst	Share	3	299.30	303.26	147	3	67/297
Burst	Lookup-69	1	913.66	1347.35	1056	3	76/297
Burst	Lookup-69	2	716.71	1416.14	1015	3	90/297
Burst	Lookup-69	3	912.42	1346.08	1056	3	80/297
Burst	Lookup-30	1	53.54	53.54	7	3	2/297
Burst	Lookup-30	2	143.41	701.03	637	3	128/297
Burst	Lookup-30	3	52.30	52.30	7	3	6/297
Burst	Local	1	4.48	6.53	7	3	6/297
Burst	Local	2	201.43	698.53	637	3	122/297
Burst	Local	3	5.72	8.24	7	3	12/297
Recovery	Share	1	305.21	335.21	187	3	122/454
Recovery	Share	2	122.16	593.86	364	3	133/454
Recovery	Share	3	304.85	334.85	187	3	123/454
Recovery	Lookup-69	1	925.33	1233.56	895	3	156/454
Recovery	Lookup-69	2	742.28	1440.00	977	3	167/454
Recovery	Lookup-69	3	924.97	1233.01	895	3	157/454
Recovery	Lookup-30	1	52.21	52.21	9	3	10/454
Recovery	Lookup-30	2	130.84	310.86	186	3	11/454
Recovery	Lookup-30	3	51.85	51.85	9	3	9/454
Recovery	Local	1	5.81	8.75	9	3	21/454
Recovery	Local	2	188.86	279.36	186	3	16/454
Recovery	Local	3	6.17	8.95	9	3	20/454
Steady	Share	1	1193.59	1538.83	1255	99	686/1564
Steady	Share	2	1068.73	1475.71	1232	99	635/1564
Steady	Share	3	1193.58	1538.95	1255	99	689/1564
Steady	Lookup-69	1	1005.76	1623.09	1548	231	596/1564
Steady	Lookup-69	2	880.91	1618.33	1546	231	635/1564
Steady	Lookup-69	3	1005.76	1623.10	1548	231	593/1564
Steady	Lookup-30	1	76.70	76.70	0	–	48/1564
Steady	Lookup-30	2	48.15	189.57	65	969	141/1564
Steady	Lookup-30	3	76.70	76.70	0	–	45/1564
Steady	Local	1	74.18	1388.10	1400	232	784/1564
Steady	Local	2	50.68	1395.18	1404	232	755/1564
Steady	Local	3	74.17	1388.23	1400	232	781/1564

Table 38. Alternative controllers under aligned deadline and cost definitions. Occupied time is in GPU seconds; cost is window scenario USD. Saving is against the historical held-out study all1 equal-window cost. The arms and historical reference use different runtimes.

Policy	strict	occupied	cost	saving (%)
B-HPA	9/9	2199.5	2.2305	-6.06
B-PRED	9/9	4007.5	4.0085	-90.62
B-MMC	9/9	2752.6	2.7861	-32.49

arms are not randomized same-runtime pairs with the frozen held-out references. This deployment confound prevents a causal comparison with H+guard.

O.2 A same-runtime supplementary comparison

The same-runtime comparison addresses the runtime confound with fresh H+guard and B-HPA runs on the known steady and recovery inputs. Both policies meet every deadline; H+guard reduces window cost by 32.60% and 49.36%, respectively, while routing more online work to the synthetic API (Table 39). This is a comparison of complete policies, including their dispatch rules. Section E provides the shared-runtime contract, all 14 run identities, phase costs, and resource–cost derivation.

The benefit depends on available latency margin and external-service prices: lower occupancy comes with more lifecycle events and queue exposure, and the API break-even multipliers are 3.08 and 11.99 at the registered GPU price. These known-input results bound the guard's resource trade-

Table 39. Same-runtime comparison: means of three fresh repeats per arm/window; all pass P1+. Cost is window scenario USD, and API share is percent of all online arrivals.

Window	Policy	Cost	API (%)
steady	B-HPA	2.4685	6.44
steady	H+guard	1.6638	51.64
recovery	B-HPA	2.0714	11.68
recovery	H+guard	1.0489	59.38

Table 40. Llama results. Completed and timely columns aggregate offline jobs only to expose missing or late work; 720 = six runs × 120 jobs. Runs, not the 720 jobs, determine the strict qualification count.

Policy	strict	completed	timely	boundary
all2	6/6	720/720	720/720	none
all1	6/6	720/720	720/720	none
H	4/6	712/720	712/720	8 unfinished
H+guard	5/6	720/720	719/720	1 late

off rather than establishing performance on new workloads.

O.3 A second model family

The Llama study uses Llama-3.1-8B-Instruct in bfloat16 with two independent TP1/PP1/DP1 instances. The 24-cell matrix contains six historical all2 gate cells reused once and 18 fresh cells, with two repeats per arm and window. All 24 cells are technically valid; the original scientific acceptance count is 23/24, whereas the stricter request-completeness count is 21/24 (Table 40). These denominators answer different questions and are not interchangeable.

H leaves four offline jobs unfinished in each recovery repeat (cells 042/043); the denominators remain 120, not the 116 observed completions. The guard completes all jobs but one recovery repeat (cell 049/r2) has only 119/120 timely offline completions. The registered Llama guard-feasibility claim therefore fails. H's recovery failure recurs across families, while the guard loses its perfect on-time record. With two repeats per arm/window, this test identifies a transfer failure but cannot establish reliability across model families.

The within-Llama model comparison is also negative. On fixed H, O_x and E_x both have equal-window $L = 0.00421052$ and occupied MAE 25.1458 GPU seconds: neither improves over the other. The registered gate requires at least 20% L reduction *and* each window's occupied MAE no worse than $O_x + 21$; passing the latter does not rescue zero improvement. The inherited 20% occupied-improvement diagnostic also fails. These locks use development median/max aggregate service lookups with identical lifecycle endpoints, not a new Llama demonstration of the Qwen endpoint-adaptation gain. M2 passes on completed-local common support (minimum 206 requests, maximum MAE 0.02964 s), but H and guard cover only 51.24% and 58.90% of all arrivals on an equal-window basis. This service-subset result does not establish whole-stream or controller fidelity; the supplement reports coverage and target disagreement separately.

O.4 Same-environment lifecycle refit

The bounded-refit study asks whether a bounded same-environment refit resolves the original Legacy-refit–Local comparison's environment caveat. It selects Local-refit by loss on two preformal development canaries; a separate preformal drift probe constrains lifecycle lower bounds. It freezes the model before CPU replay and evaluates all 36 existing physical records without new GPU runs. On fixed H, equal-window L falls from 0.01352738 to 0.01284857, a 5.02% reduction below the registered 10% threshold, while every window satisfies the occupied-MAE safety line Local-refit $\leq$ Local + 21. Steady-window L worsens despite the aggregate reduction. The result is a scientific negative, not evidence that refitting has no effect. Only lifecycle scalars are eligible for fitting; service, setup, and guard safety remain fixed. The researchers already knew the held-out study's aggregate results, although the fitting code never consumed formal outcomes. This is a bounded retrospective replay, not a newly blinded experiment, and it does not erase the historical Legacy-refit–Local caveat.

O.5 Second arrival source and a load boundary

The Azure conversation source associated with Splitwise (Patel et al., 2024) contains 19,366 rows over about 58.4 minutes. The Azure load study uses timestamps from its first and last non-overlapping 25-minute windows. Replay uses fixed-corpus prompts and controlled token budgets, with the held-out study synthetic API contract. This does not replay the original prompt content or generated responses. The original full-strength all2 gate is technically valid but scientifically negative in all six runs (Table 12); the other 18 planned policy cells are `NOT_RUN_GATE_STOP`, not missing successes. Equal-window mean online timely completion is only 66.2% (39.6–93.1% across runs), and offline timely completion is 29.9%. No feasible-saving claim is made at that intensity.

The sequential load-thinning amendment fixes integer SHA thinning at online intensities 0.75, 0.50, then optionally 0.25. It preserves retained row identities, arrival times, deadlines, and all 120 offline jobs per window. At 0.75, all work eventually completes, but only 113/120 front-window and 79/120 back-window offline jobs are timely in each repeat. At 0.50, all six all2 gate runs pass: this is the first feasible

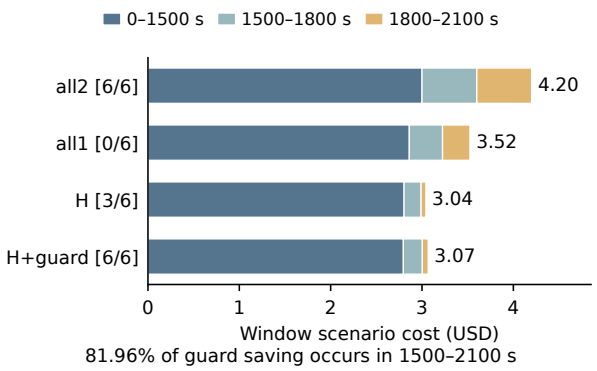


Figure 4. Thinned-Azure study costs by window-time segment. Bars average three repeats per window, then weight the two windows equally; brackets give strict passes out of six. Lower cost in failed runs is not feasible saving. Of the guard's saving against all2, 81.96% occurs in the final 600 seconds.

tested level. The subsequent 24 fresh runs use that locked level, four policies, two windows, and three repeats. The six successful gate runs are not merged into the fresh all2 denominator. The optional 0.25 level was not added.

The fresh comparison separates cheap failure from feasible saving (Figure 4 and Table 13). all1 qualifies in 0/6 runs and H in 3/6: H's front repeats 1 and 3 each censor two offline jobs, while back repeat 2 censors four. These eight unfinished jobs fail the strict completion criterion despite the recorded `contract_feasible` flag. The guard qualifies in 6/6 and reduces equal-window occupied time from 4200.000 to 2992.407 GPU seconds (28.75%), window cost from 4.2000 to 3.0675 (26.96%), and full-deployment cost from 4.3204 to 3.1366 (27.40%). These percentages are ratios of equal-window means, not averages of paired per-run percentages. The official aggregate `NEGATIVE_RESULT`, driven by all1, is preserved. The time decomposition further limits the economic interpretation: guard cost falls only 6.81% during the first 1,500 seconds, and about 81.96% of the absolute whole-window saving arises in the final 600 seconds. The 26.96% headline must therefore not be extrapolated to sustained high-load serving efficiency. Figure 4 makes the concentration explicit; the supplement retains the numerical segment ledger.

Thinning reuses observed source windows; intensity was selected by the registered all2 feasibility gate, not outcome-independent sampling. The result is therefore load-conditioned, not a new held-out trace. Offline arrivals continue during $[1500, 1800)$, and the back window retains an online arrival at the closed 1500-second endpoint, so source-window timing and ledger segments are reported separately. These results do not estimate Azure's production QoS, required fleet size, model quality, energy use, or bills.

Retrospective predictor screening. We replay the 24 fresh thinned-Azure runs with the archived Share/Lookup-69/Lookup-30/Earlier-local/Legacy-refit model bank, original policy code, and fixed guard estimates. Eight policy/window predictions per model are reused across three repeats, yielding 40 distinct predictions and 120 comparisons. Raw request and resource reconstruction matches the historical records; strict P1+ is recomputed from every request's deadline and observation cutoff. These predictions are generated after the physical study, without fitting its outcomes.

Of the 24 physical runs, 15 satisfy P1+ and nine fail. Lookup-69, Lookup-30, Earlier-local, and Legacy-refit each agree on 21/24, but Lookup-69 falsely accepts three failures whereas Lookup-30/Earlier-local/Legacy-refit reject three successes. Share agrees on 9/24, rejecting all 15 successes. Their window-cost MAEs are 0.0909, 0.0419, 0.0536, 0.0523, and 1.6775 scenario USD for Lookup-69, Lookup-30, Earlier-local, Legacy-refit, and Share, respectively. H succeeds in one front-window repeat and two back-window repeats. A fixed binary prediction cannot match every one of these mixed repeats; no majority outcome replaces the individual runs. This second arrival source illustrates why error direction and physical variation must accompany total agreement. The archived Earlier-local model here uses 25.196 s startup, distinct from the deployment-adapted Local in the main study; all original model fields are retained in `artifact/azure_diagnostic/`.

REFERENCES

Agrawal, A., Kedia, N., Mohan, J., Panwar, A., Kwatra, N., Gulavani, B. S., Ramjee, R., and Tumanov, A. Vidur: A Large-Scale Simulation Framework for LLM Inference. In *Proceedings of Machine Learning and Systems*, volume 6, pp. 351–366, 2024. URL https://proceedings.mlsys.org/paper_files/paper/2024/hash/b74a8de47d2b3c928360e0a011f48351-Abstract-Conference.html.

Patel, P., Choukse, E., Zhang, C., Shah, A., Goiri, Í., Maleki, S., and Bianchini, R. Splitwise: Efficient Generative LLM Inference Using Phase Splitting. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*, pp. 118–132, 2024. doi: 10.1109/isca59077.2024.00019. URL https://doi.org/10.1109/isca59077.2024.00019.